



University
of Exeter

TM & NLP (COMM040)

Dr. Rodrigo Wilkens

Intended Learning Outcomes



- Apply NLP methods to realistic data sets, demonstrating the ability to clean, transform, analyse and interpret the output of NLP algorithms.
- Understand the strengths, weaknesses and biases of different NLP methods
- Know about recent advances and applications of NLP methods, their impact and real-world applications.
- Use modern NLP techniques to derive insight from unstructured data
- Communicate the pros and cons and trade-offs of different approaches to text analysis

Outline

- Text Encoding
- Classification Approach
- Hybrid Architectures
- PLM
- Parameter-Efficient Fine-Tuning

Outline

- Text Encoding
- Classification Approach
- Hybrid Architectures
- PLM
- Parameter-Efficient Fine-Tuning

University
of Exeter

- [illegible]

[illegible]

Text Encoding

The dog bit the man.

1	0	1	0	0	0	0	0	0	0
0	0	1	1	0	0	0	0	0	0
0	0	0	1	0	0	0	0	0	0
0	0	0	0	0	1	0	0	0	1
0	0	0	0	1	0	0	0	0	0
0	0	1	1	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	1	0	0	0
1	0	0	0	0	0	0	0	0	0
0	0	1	0	0	1	0	0	1	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	1

No Context: Words are treated as isolated islands.

The man bit the dog.

1	0	1	0	0	0	0	0	0	0
0	0	1	1	0	0	0	0	0	0
0	0	0	1	0	0	0	0	0	0
0	0	0	0	0	1	0	0	0	1
0	0	0	0	1	0	0	0	0	0
0	0	1	1	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	1	0	0	0
1	0	0	0	0	0	0	0	0	0
0	0	1	0	0	1	0	0	1	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	1

No Order: Syntax and grammar are completely erased.

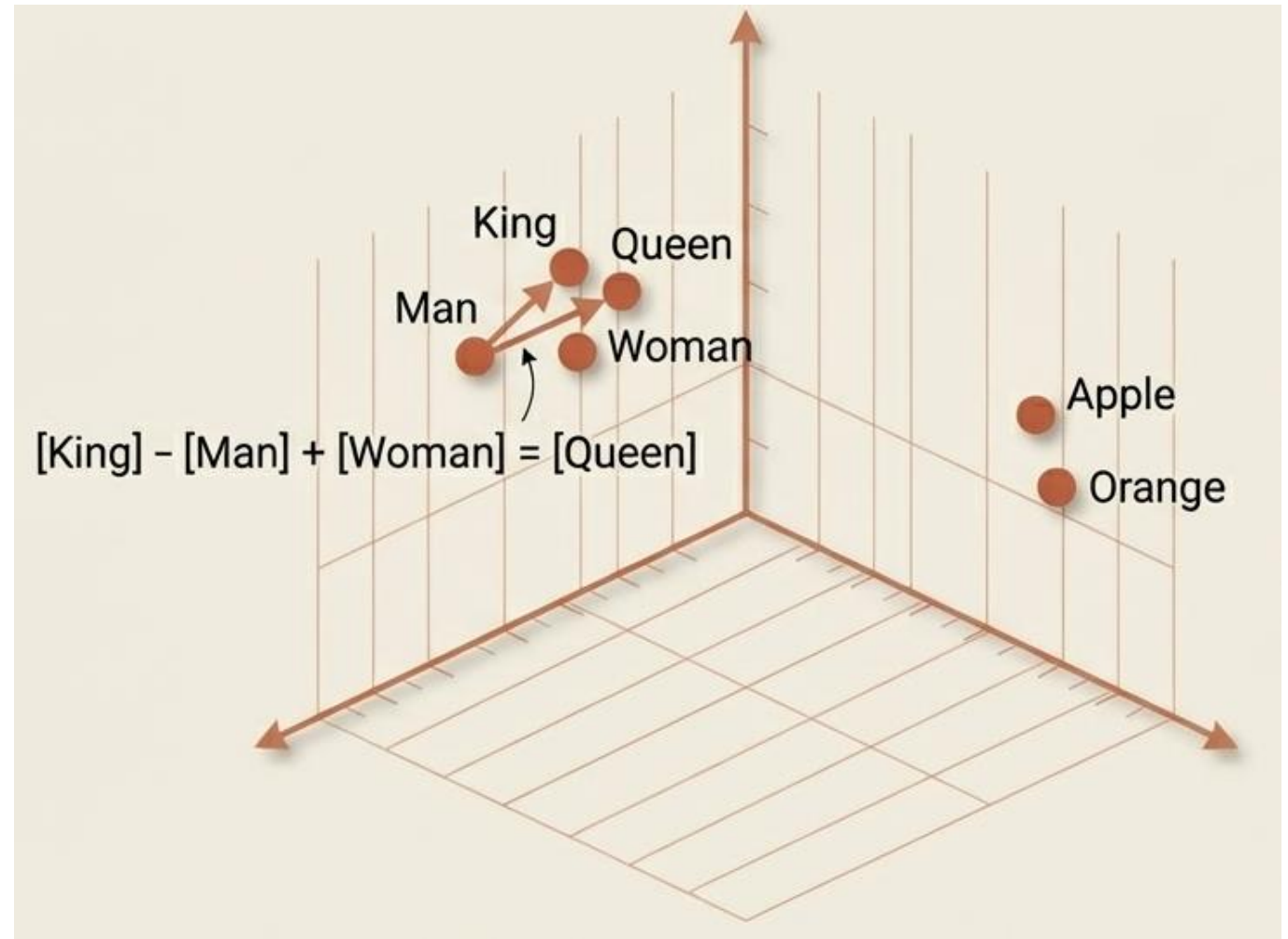
=

No Order: Syntax and grammar are completely erased.

Curse of Dimensionality: Massive, inefficient matrices that break down as vocabulary grows.

Text Encoding

- Static Embeddings
 - Word2Vec
 - Glove
 - FastText



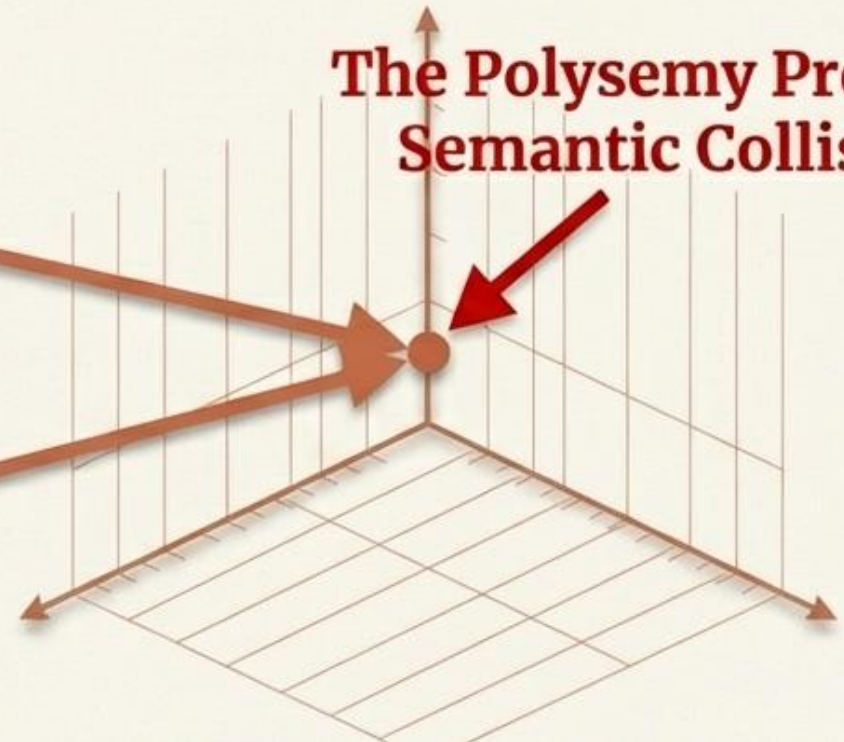
Text Encoding

- Static Embeddings

I ate a crisp **Apple**.

Apple released a
new phone.

The Polysemy Problem:
Semantic Collision.



Text Encoding

- Contextualized Embeddings



Text Encoding

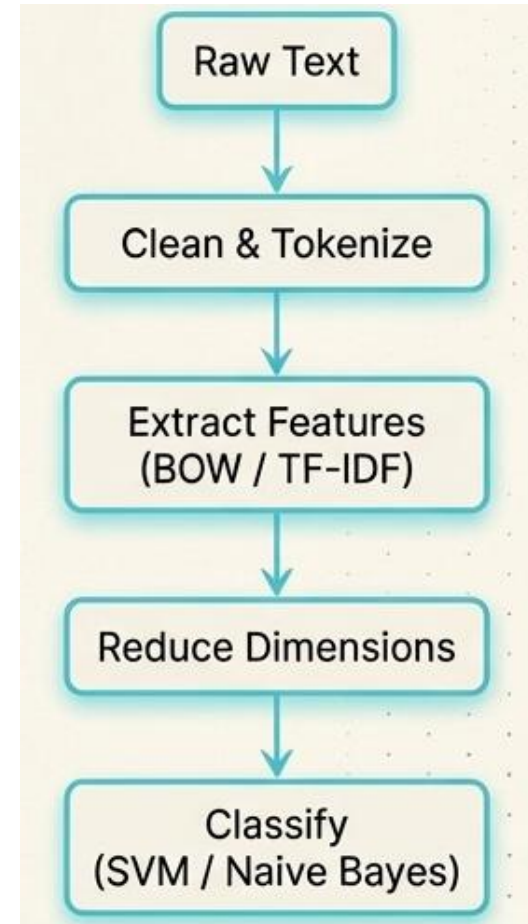
- Which information is progressively added when moving from BoW → static embeddings → contextual embeddings?

Outline

- Text Encoding
- **Classification Approach**
- Hybrid Architectures
- PLM
- Parameter-Efficient Fine-Tuning

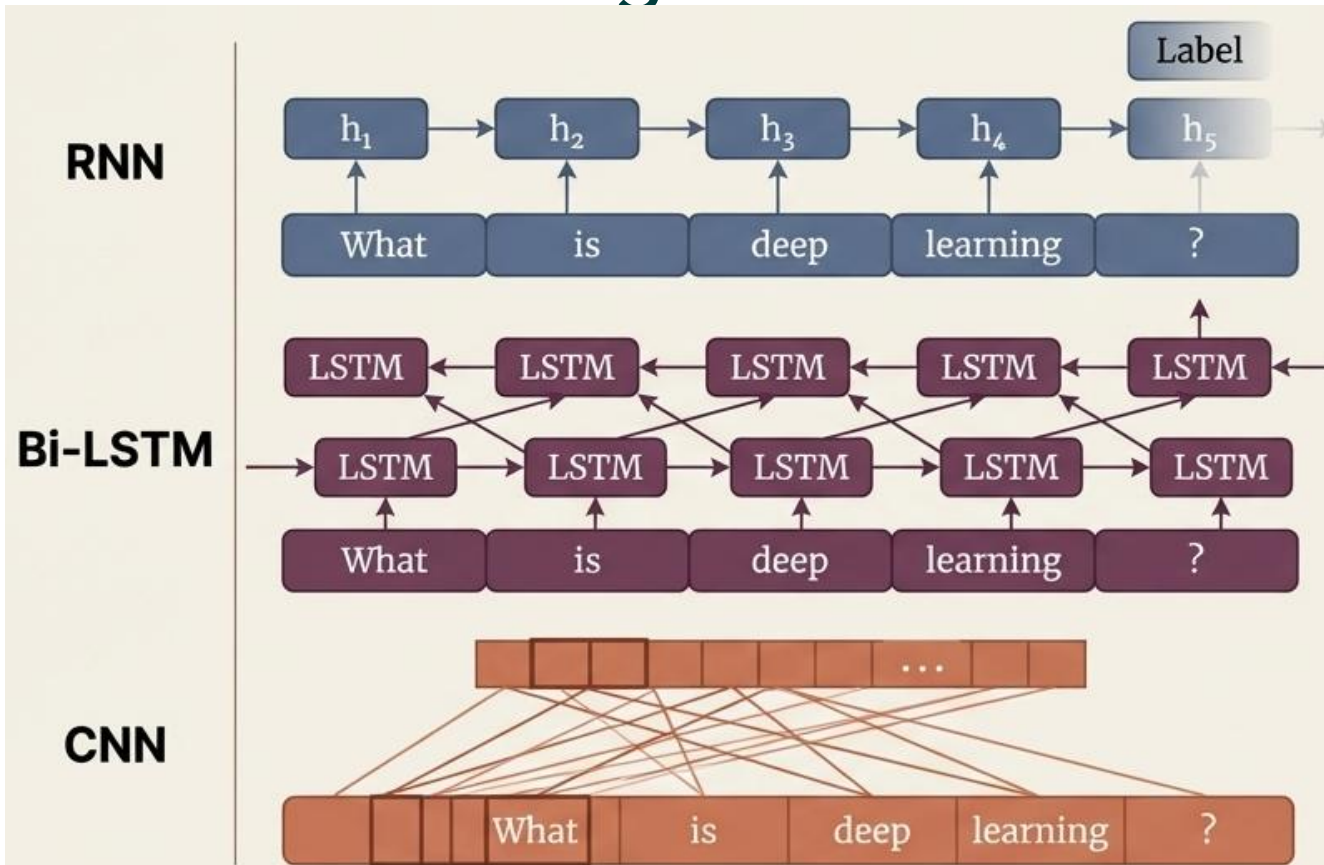
Classification Approach

- Context-Independent
 - SVM
 - Random Forest
 - XGBoost



Classification Approach

- Static Embeddings
 - Gradient Vanishing



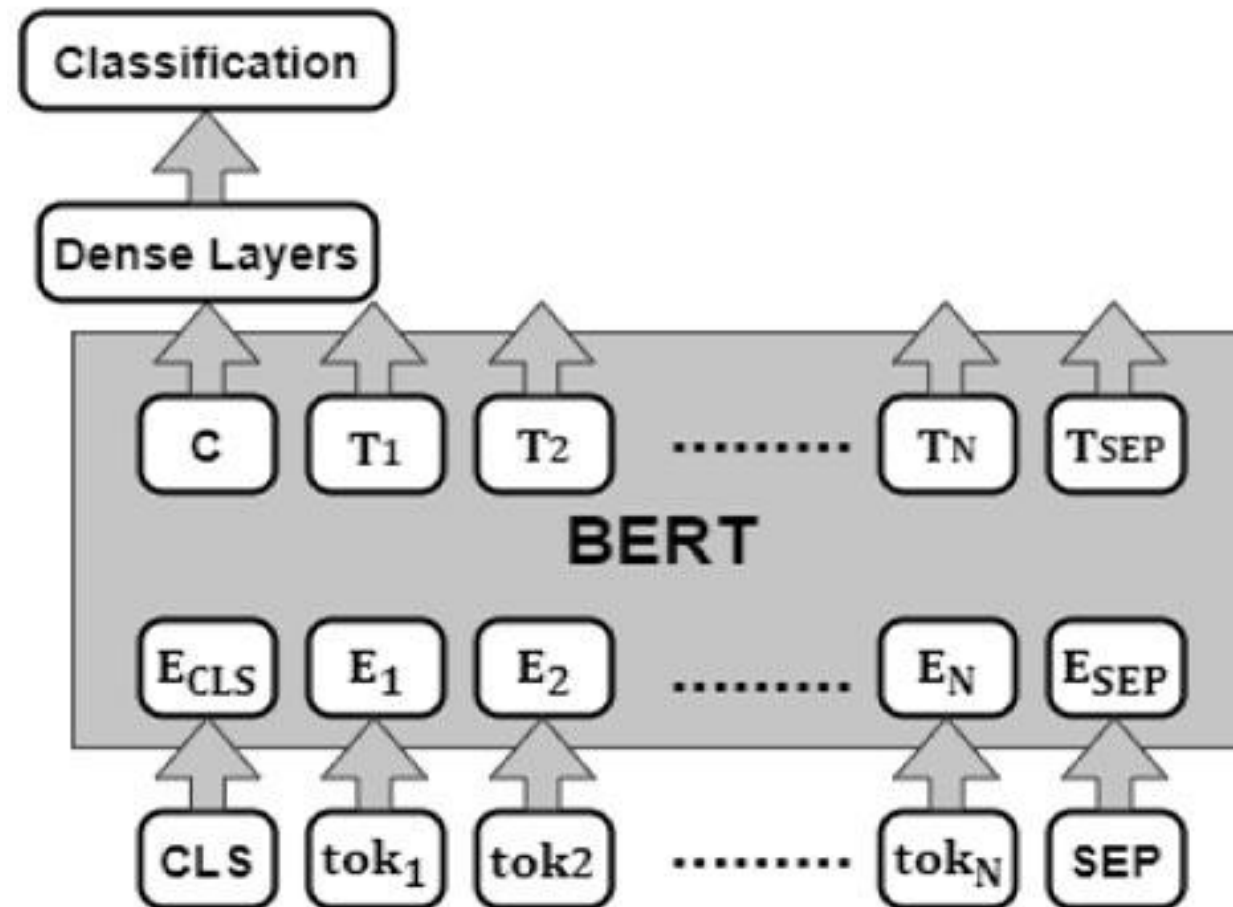
RNN: Reads sequentially left-to-right, but forgets early words (Vanishing Gradient).

Bi-LSTM: Reads in both directions simultaneously, keeping long-term memory.

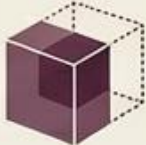
CNN: Great at local n-gram pattern extraction, but poor at global sentence structure.

Classification Approach

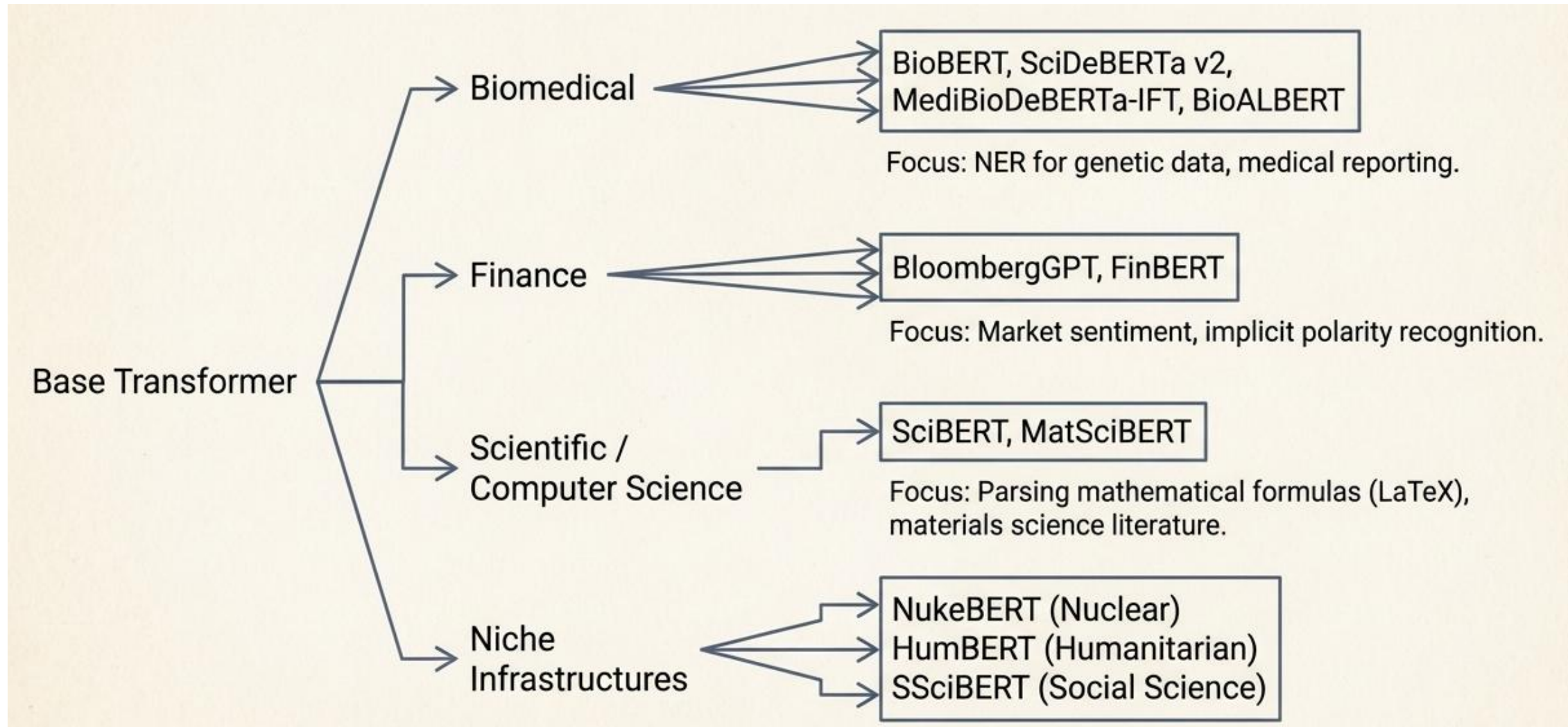
- Contextualised Embeddings



Text Encoding and Classification

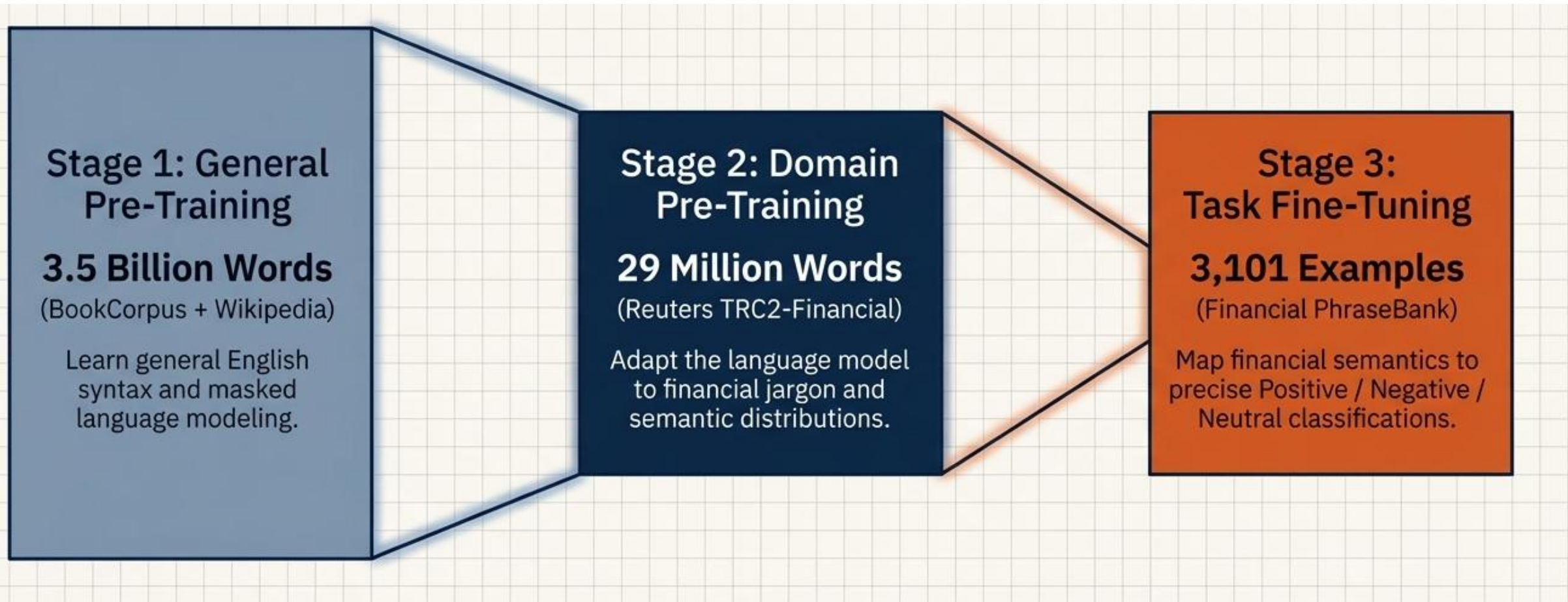
	Count-Based	Static Neural	Sequential Contextual	Transformer Contextual
Core Concept	Frequency counting 	Predicting neighbors 	Sequence memory 	Global self-attention 
Dimensionality	Sparse (Size of vocabulary) 	Dense (Fixed ~300) 	Dense (Dynamic) 	Dense (Massive context window) 
Context-Awareness	None (Order lost) 	Static (One vector per word) 	Dynamic (Directional context) 	Omnidirectional (Simultaneous context) 
Example Models	TF-IDF, BoW 	Word2Vec, GloVe 	ELMo, Bi-LSTM 	BERT, GPT-4, Llama 

Domain Specific Models



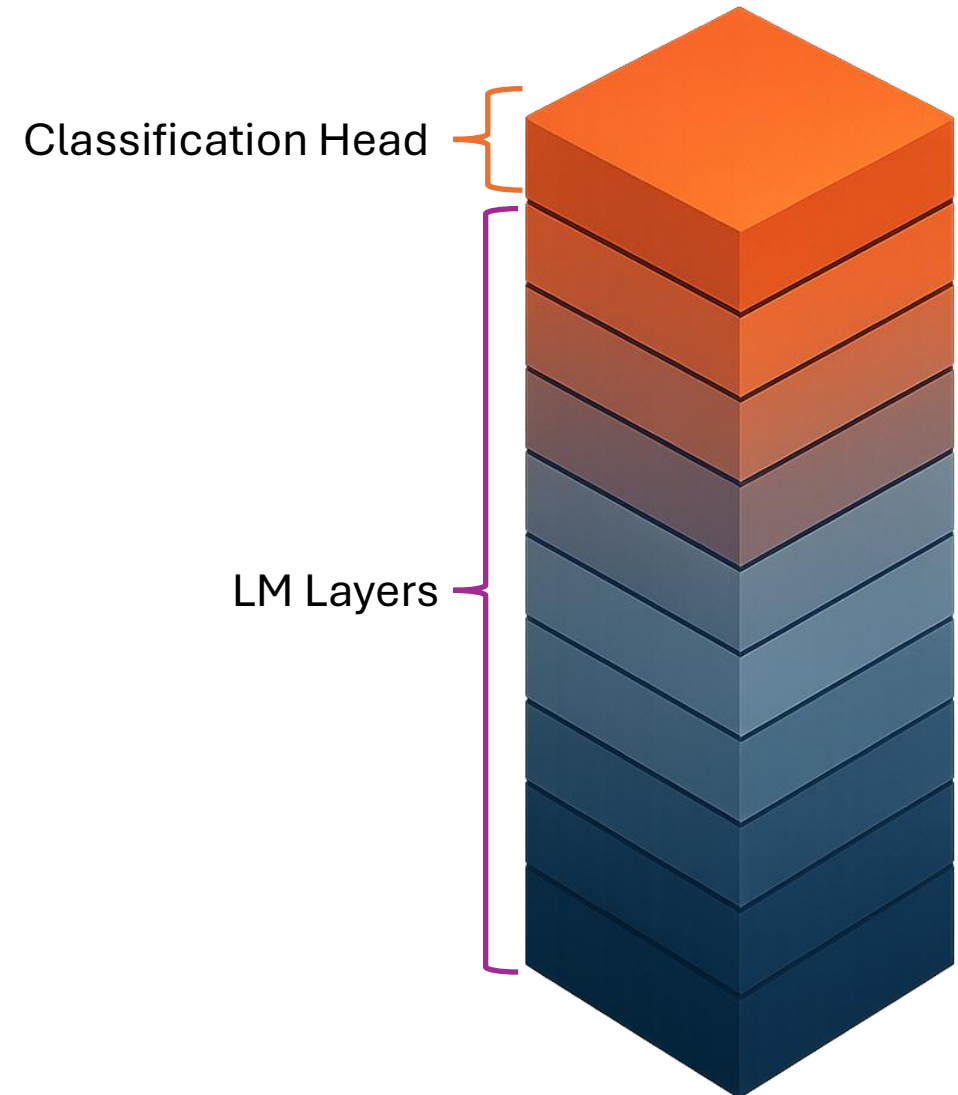
Domain Specific Models

- FinBERT



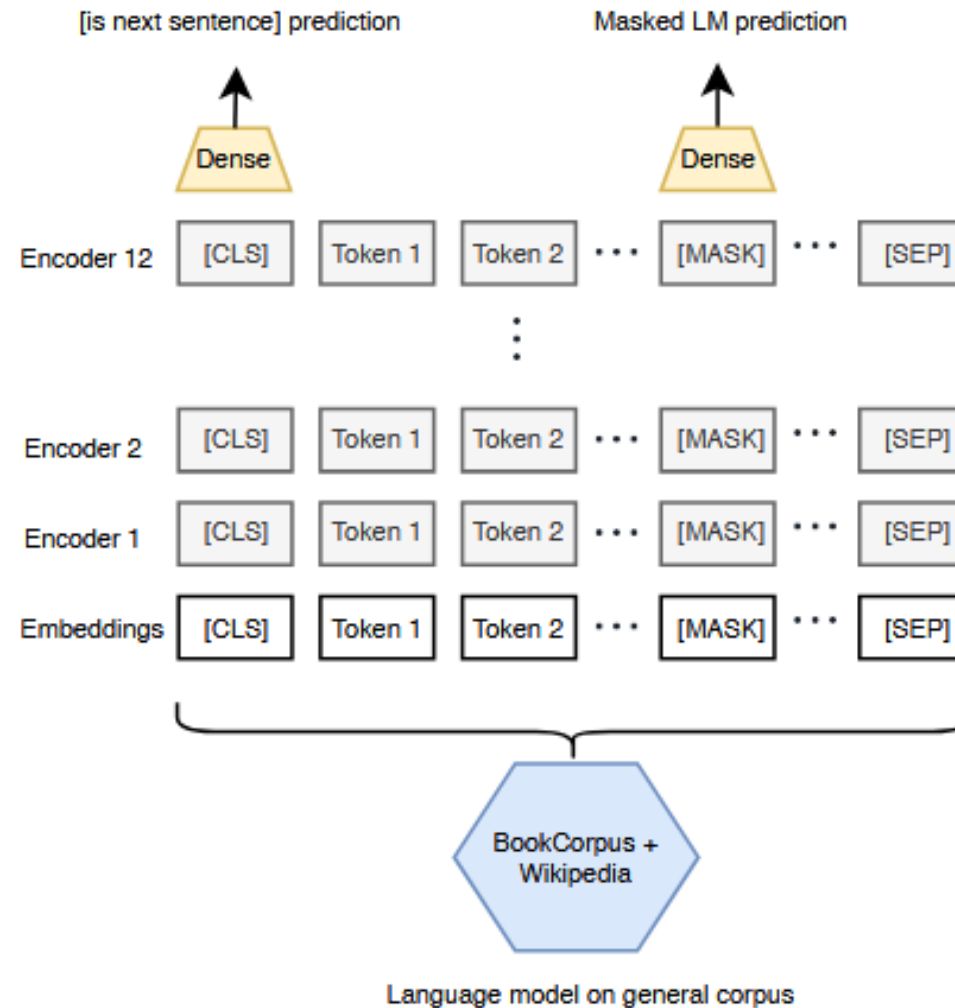
Domain Specific Models

- FinBERT
 - Architecture: BERT
 - Original Model: Bert-base
 - Epochs: 6
 - Dropout: 0.1
 - Learning Rate: $2e - 5$
 - Discriminative fine-tuning: 0.85



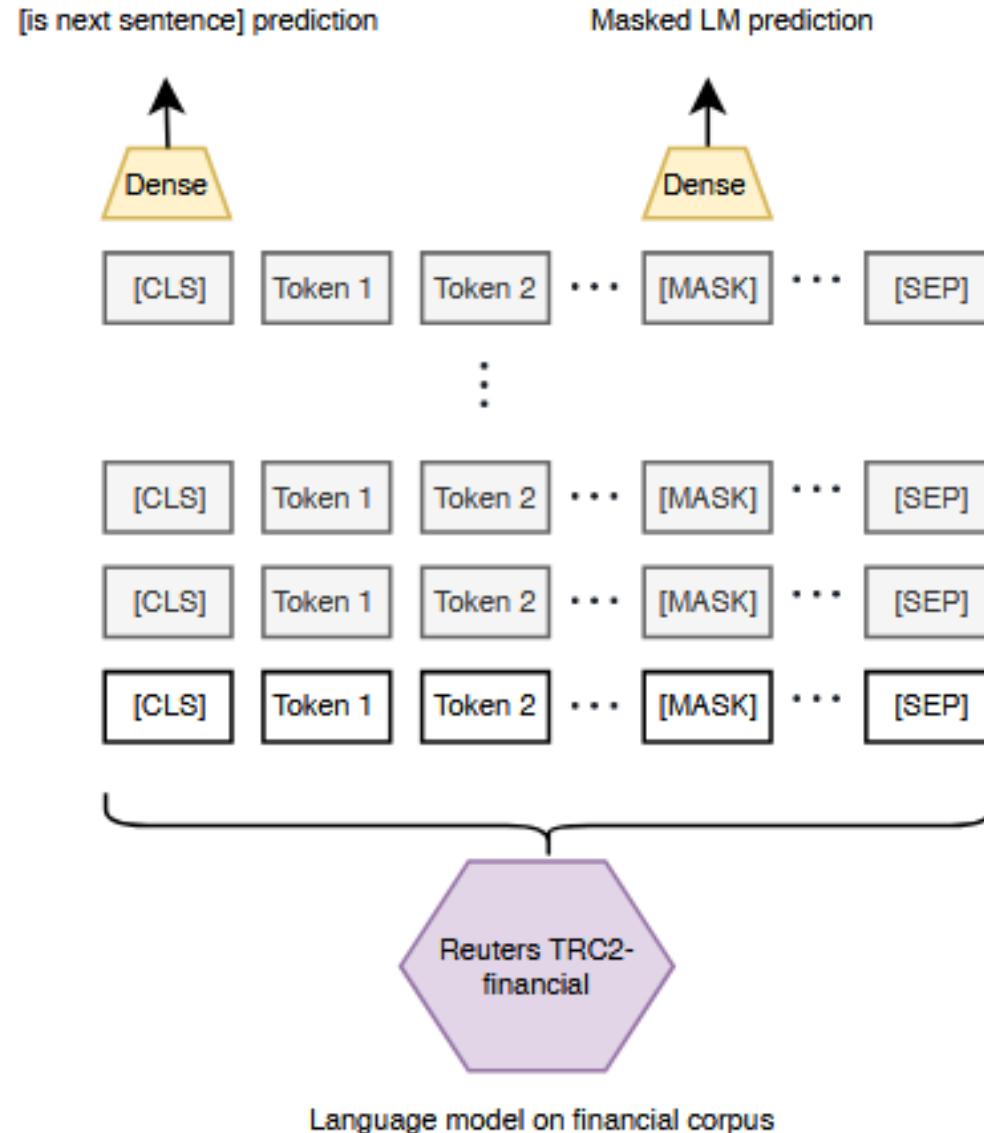
Domain Specific Models

- FinBERT
 - Step 1:
 - BERT



Domain Specific Models

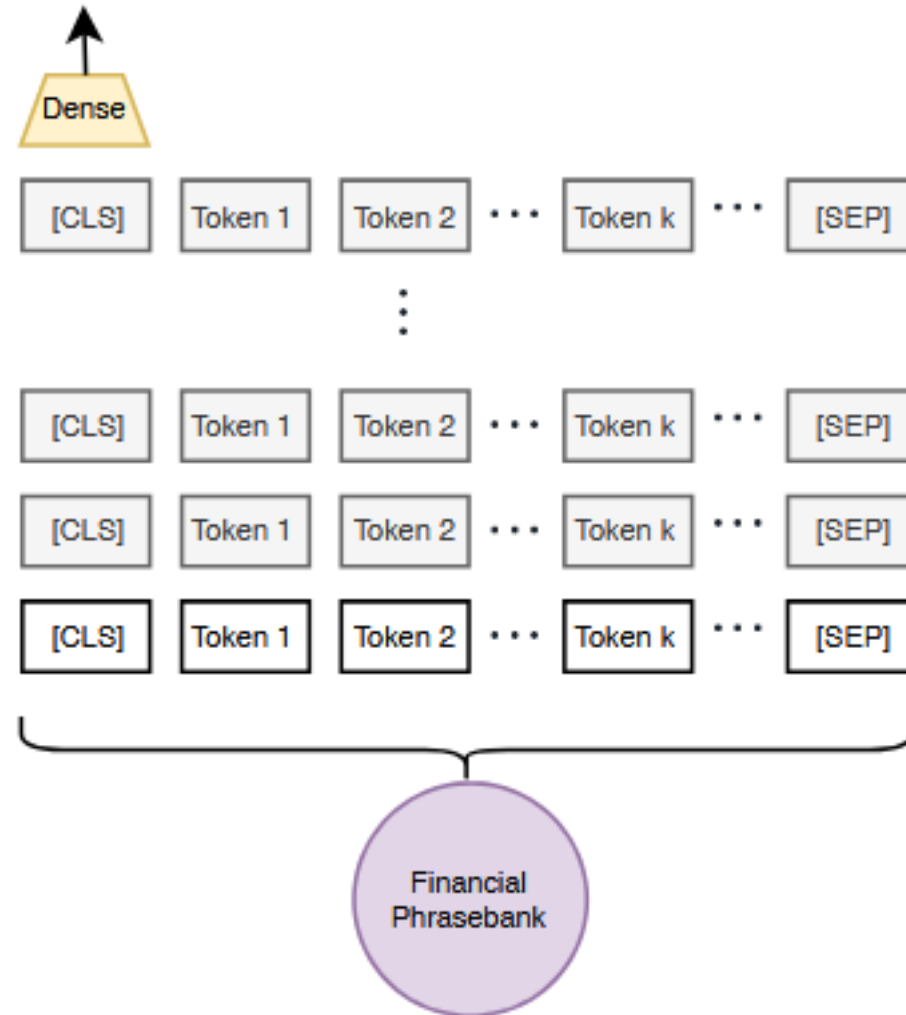
- FinBERT
 - Step 2:
 - Domain Adaptation



Domain Specific Models

- FinBERT
 - Step 3:
 - Task Specialisation

Sentiment prediction



Classification model on financial sentiment dataset

Domain Specific Models

- FinBERT

Performance on different encoder layers used for classification

Layer for classification	Loss	Accuracy	F1 Score
Layer-1	0.65	0.76	0.77
Layer-2	0.54	0.78	0.78
Layer-3	0.52	0.76	0.77
Layer-4	0.48	0.80	0.77
Layer-5	0.52	0.80	0.80
Layer-6	0.45	0.82	0.82
Layer-7	0.43	0.82	0.83
Layer-8	0.44	0.83	0.81
Layer-9	0.41	0.84	0.82
Layer-10	0.42	0.83	0.82
Layer-11	0.38	0.84	0.83
Layer-12	0.37	0.86	0.84

Domain Specific Models

• FinBERT

Performance on different encoder layers used for classification

Layer for classification	Loss	Accuracy	F1 Score
Layer-1	0.65	0.76	0.77
Layer-2	0.54	0.78	0.78
Layer-3	0.52	0.76	0.77
Layer-4	0.48	0.80	0.77
Layer-5	0.52	0.80	0.80
Layer-6	0.45	0.82	0.82
Layer-7	0.43	0.82	0.83
Layer-8	0.44	0.83	0.81
Layer-9	0.41	0.84	0.82
Layer-10	0.42	0.83	0.82
Layer-11	0.38	0.84	0.83
Layer-12	0.37	0.86	0.84

Performance on starting training from different layers

First layer unfreezed	Loss	Accuracy	Training time
Embeddings layer	0.37	0.86	332s
Layer-1	0.39	0.83	302s
Layer-2	0.39	0.83	291s
Layer-3	0.38	0.83	272s
Layer-4	0.38	0.82	250s
Layer-5	0.40	0.83	240s
Layer-6	0.40	0.81	220s
Layer-7	0.39	0.82	205s
Layer-8	0.39	0.84	188s
Layer-9	0.39	0.84	172s
Layer-10	0.41	0.84	158s
Layer-11	0.45	0.82	144s
Layer-12	0.47	0.81	133s
Classification layer	1.04	0.52	119s

Classification Approach



- [menti.com](https://www.menti.com)

Sentence Similarity

- How do you calculate semantic textual similarity (STS) between two sentences?

Sentence Similarity

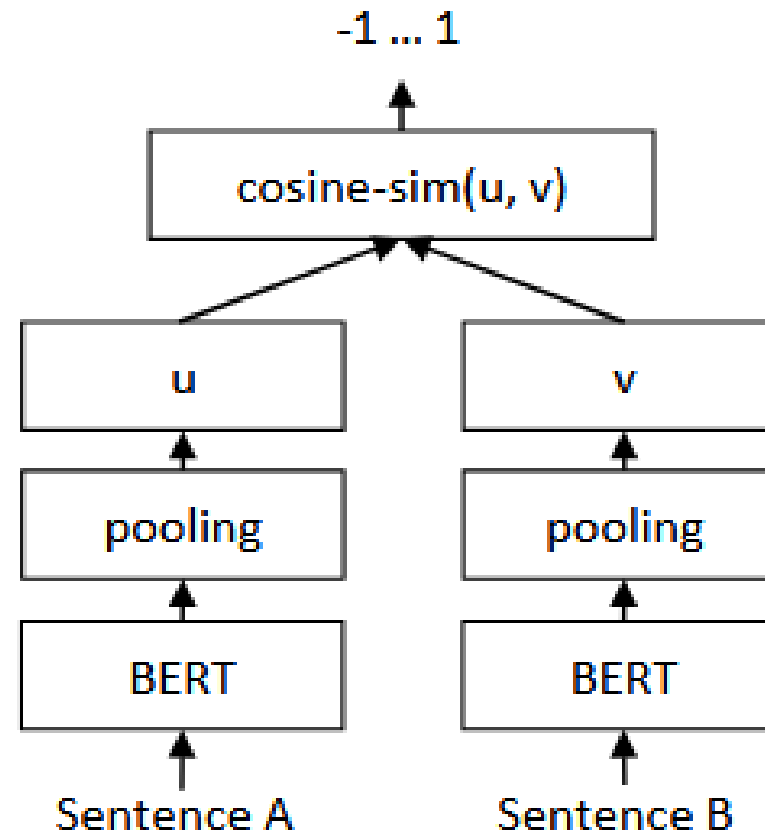
- Semantic relatedness

Model	Avg.
Avg. GloVe embeddings	61.32
Avg. BERT embeddings	54.81
BERT CLS-vector	29.19
InferSent - Glove	65.01
Universal Sentence Encoder	71.22
SBERT-NLI-base	74.89
SBERT-NLI-large	76.55
SRoBERTa-NLI-base	74.21
SRoBERTa-NLI-large	76.68

The Spearman's rank correlation between the cosine-similarity of the sentence embeddings and the gold labels.

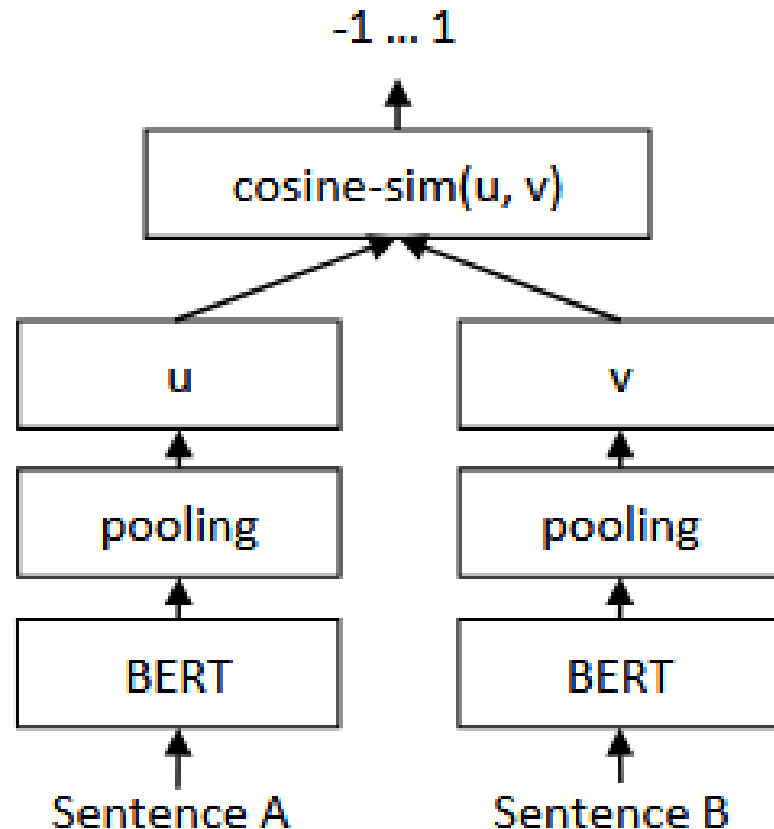
Sentence-BERT

- Sentence Similarity
(Regression)



Sentence-BERT

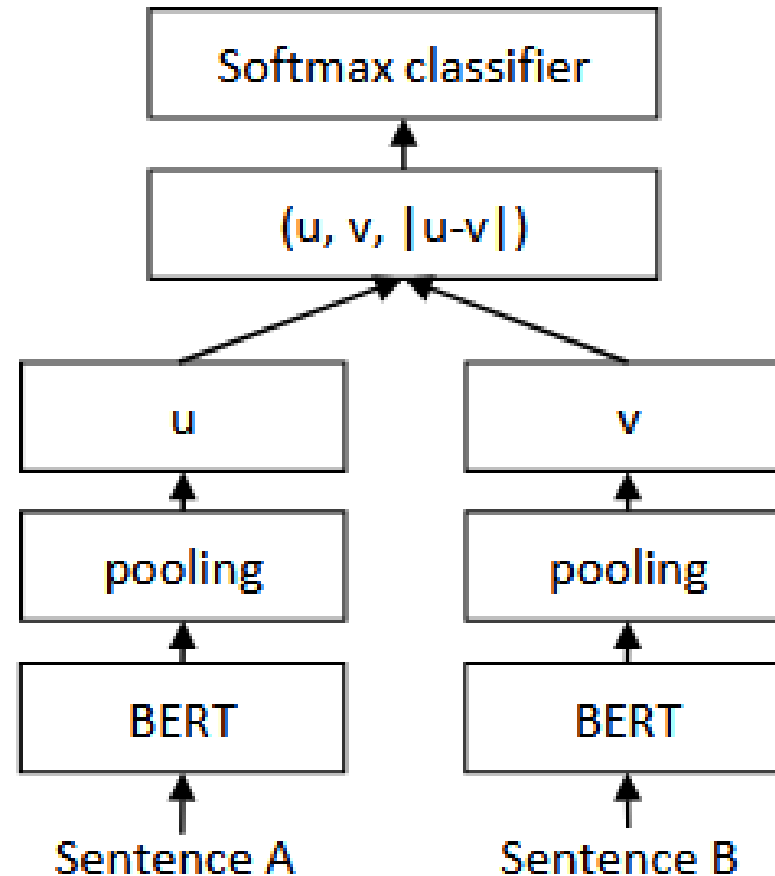
- Sentence Similarity
(Regression)



Model	Spearman
<i>Not trained for STS</i>	
Avg. GloVe embeddings	58.02
Avg. BERT embeddings	46.35
InferSent - GloVe	68.03
Universal Sentence Encoder	74.92
SBERT-NLI-base	77.03
SBERT-NLI-large	79.23
<i>Trained on STS benchmark dataset</i>	
BERT-STSB-base	84.30 ± 0.76
SBERT-STSB-base	84.67 ± 0.19
SRoBERTa-STSB-base	84.92 ± 0.34
BERT-STSB-large	85.64 ± 0.81
SBERT-STSB-large	84.45 ± 0.43
SRoBERTa-STSB-large	85.02 ± 0.76

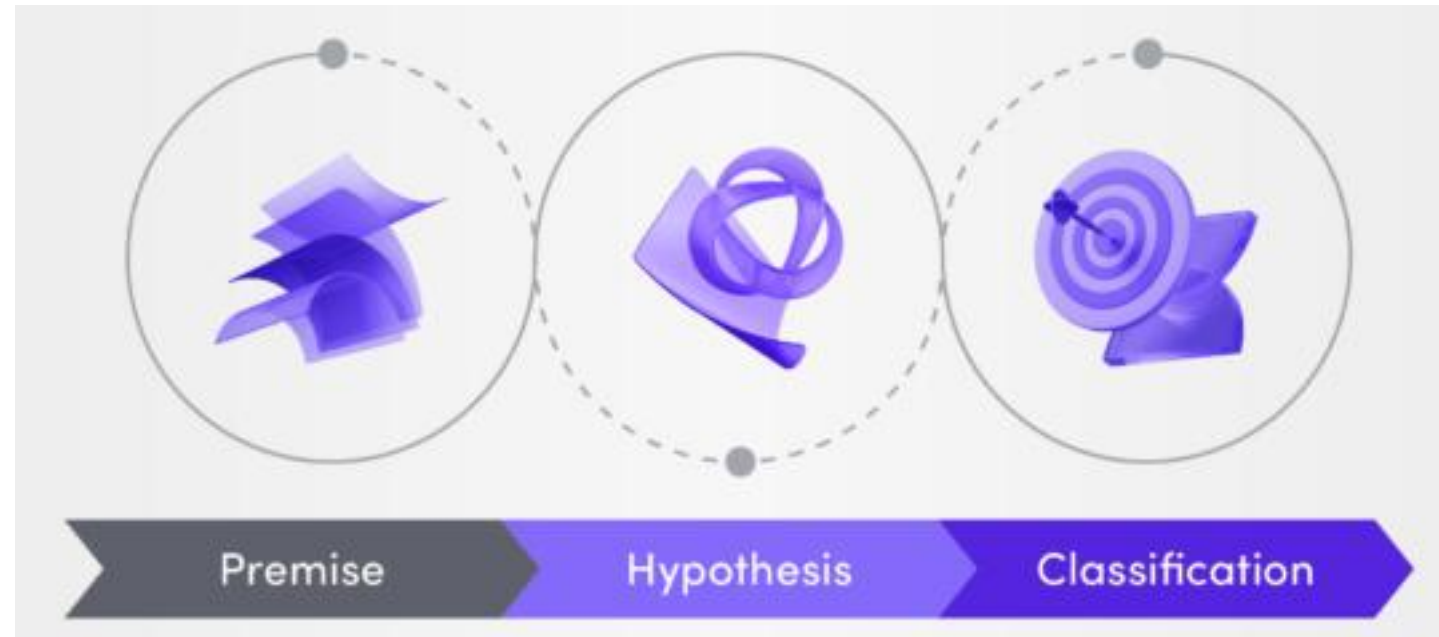
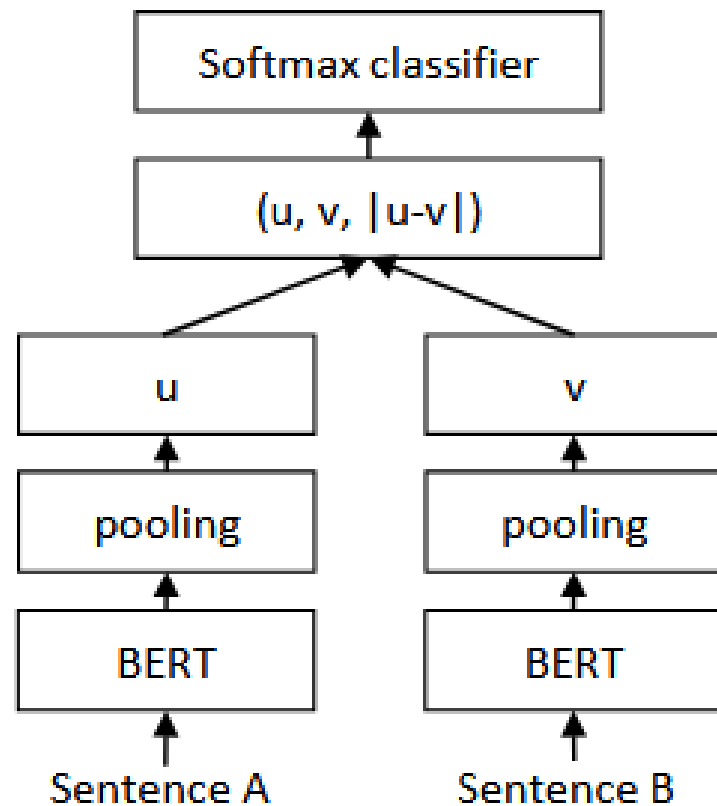
Sentence-BERT

- NLI
(Classification)



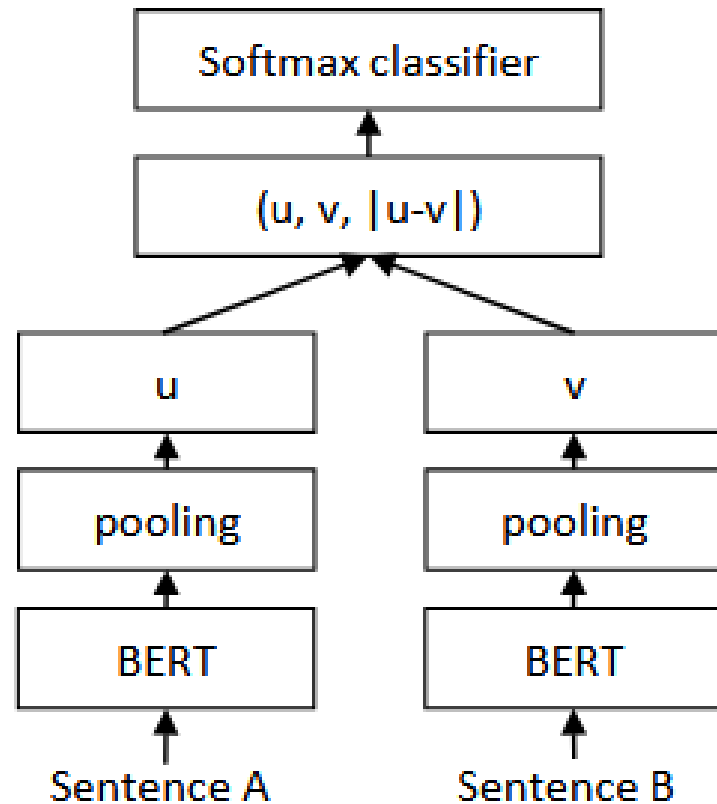
Sentence-BERT

- NLI
(Classification)



Sentence-BERT

- NLI
(Classification)



Premise: An adult dressed in black holds a stick.

Hypothesis: An adult is walking away, empty-handed.

Label: contradiction

Explanation: Holds a stick implies using hands so it is not empty-handed.

Premise: A child in a yellow plastic safety swing is laughing as a dark-haired woman in pink and coral pants stands behind her.

Hypothesis: A young mother is playing with her daughter in a swing.

Label: neutral

Explanation: Child does not imply daughter and woman does not imply mother.

Premise: A man in an orange vest leans over a pickup truck.

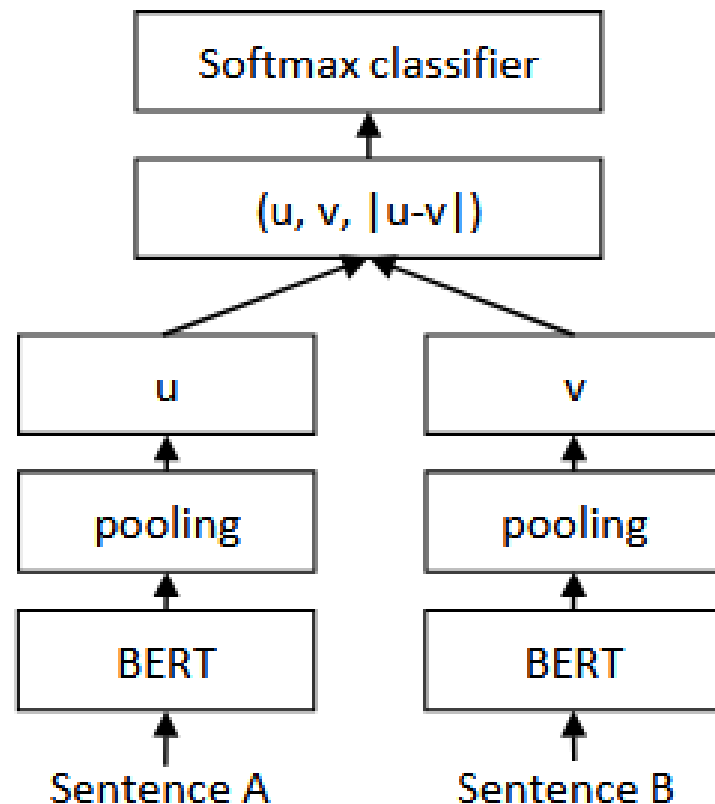
Hypothesis: A man is touching a truck.

Label: entailment

Explanation: Man leans over a pickup truck implies that he is touching it.

Sentence-BERT

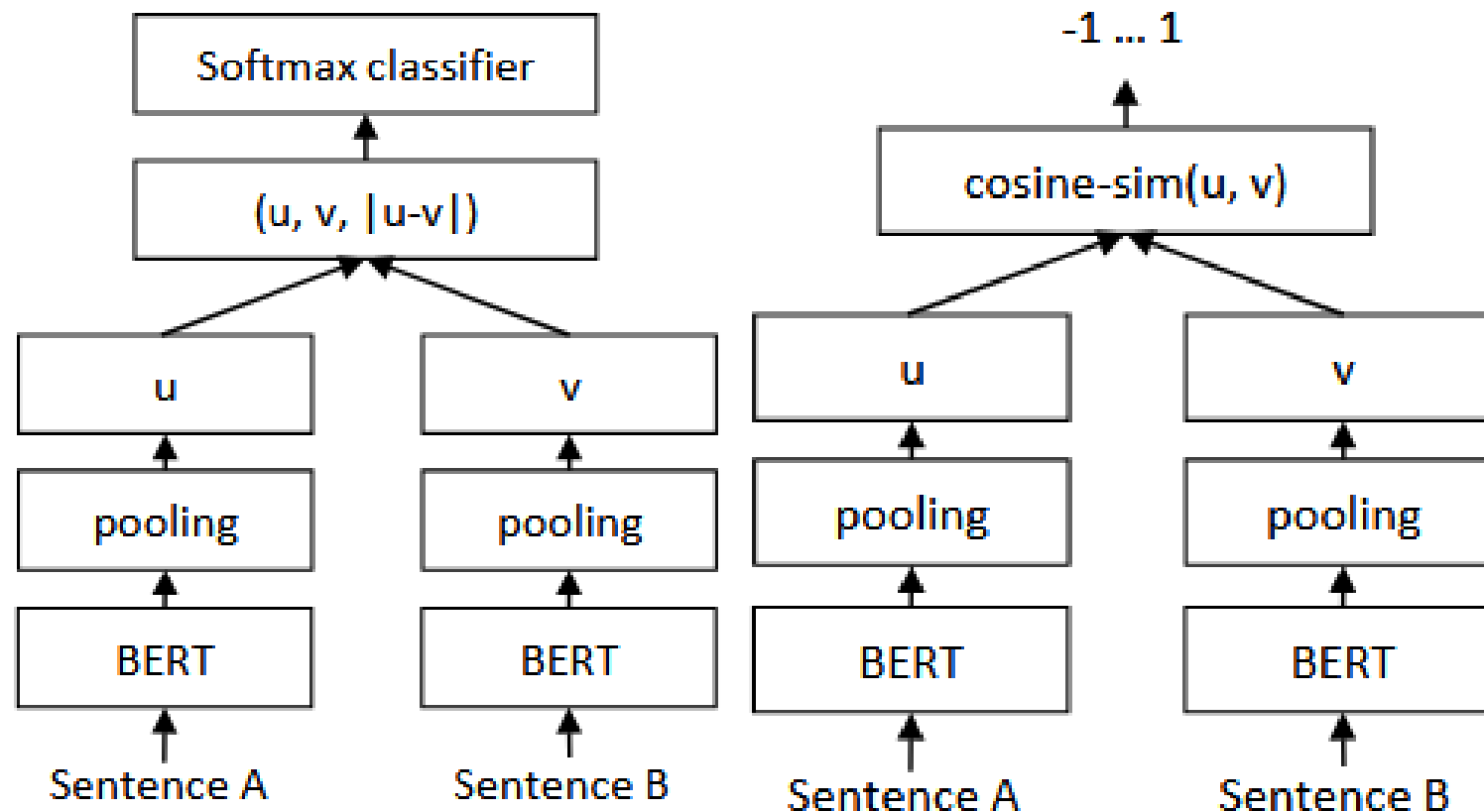
- NLI
(Classification)



Model	r	ρ
<i>Unsupervised methods</i>		
tf-idf	46.77	42.95
Avg. GloVe embeddings	32.40	34.00
InferSent - GloVe	27.08	26.63
<i>10-fold Cross-Validation</i>		
SVR (Misra et al., 2016)	63.33	-
BERT-AFS-base	77.20	74.84
SBERT-AFS-base	76.57	74.13
BERT-AFS-large	78.68	76.38
SBERT-AFS-large	77.85	75.93

Sentence-BERT

- Sentence Similarity
(Classification)



Model	Spearman
<i>Trained on NLI data + STS benchmark data</i>	
BERT-NLI-STSB-base	88.33 $\pm$ 0.19
SBERT-NLI-STSB-base	85.35 $\pm$ 0.17
SRoBERTa-NLI-STSB-base	84.79 $\pm$ 0.38

Sentence-BERT

- SentEval (regression architecture)
 - Sentiment prediction
 - movie reviews (MR)
 - customer product (CR)
 - Stanford Sentiment Treebank (SST)
 - Subjectivity prediction (SUBJ)
 - Opinion polarity (MPQA)
 - Question-type (TREC)
 - Paraphrase (MRPC)

Sentence-BERT

- SentEval (regression architecture)

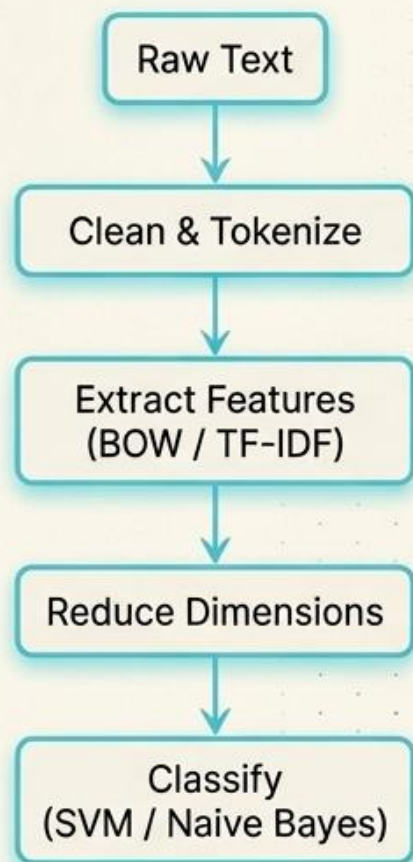
Model	MR	CR	SUBJ	MPQA	SST	TREC	MRPC	Avg.
Avg. GloVe embeddings	77.25	78.30	91.17	87.85	80.18	83.0	72.87	81.52
Avg. fast-text embeddings	77.96	79.23	91.68	87.81	82.15	83.6	74.49	82.42
Avg. BERT embeddings	78.66	86.25	94.37	88.66	84.40	92.8	69.45	84.94
BERT CLS-vector	78.68	84.85	94.21	88.23	84.13	91.4	71.13	84.66
InferSent - GloVe	81.57	86.54	92.50	90.38	84.18	88.2	75.77	85.59
Universal Sentence Encoder	80.09	85.19	93.98	86.70	86.38	93.2	70.14	85.10
SBERT-NLI-base	83.64	89.43	94.39	89.86	88.96	89.6	76.00	87.41
SBERT-NLI-large	84.88	90.07	94.52	90.33	90.66	87.4	75.94	87.69

Outline

- Text Encoding
- Classification Approach
- **Hybrid Architectures**
- PLM
- Parameter-Efficient Fine-Tuning

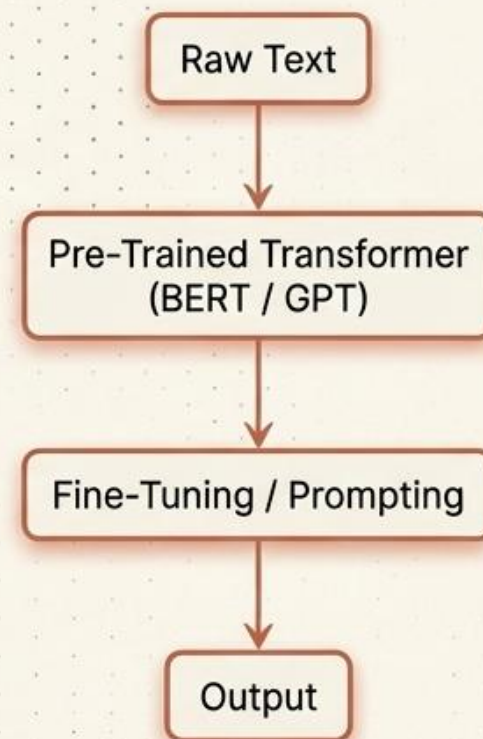
The Traditional Pipeline

(Modular, feature-engineered, heavily reliant on dimensionality reduction)



The Modern Paradigm

(End-to-end, representation-learning, contextually aware)



Hybrid Architectures

- 2 Textual Sources

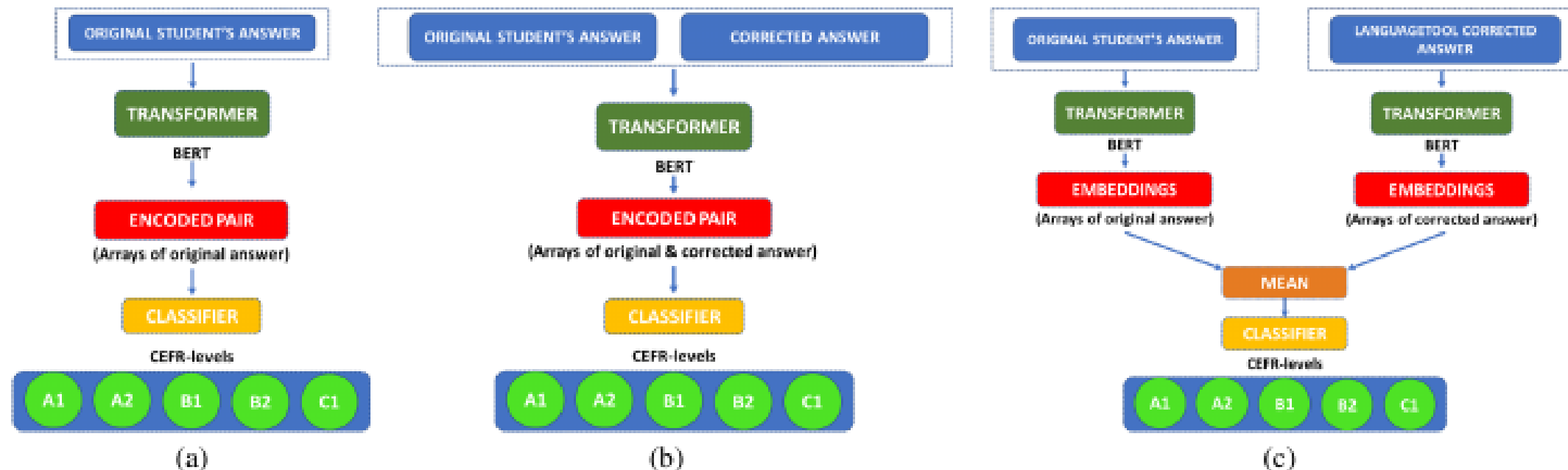
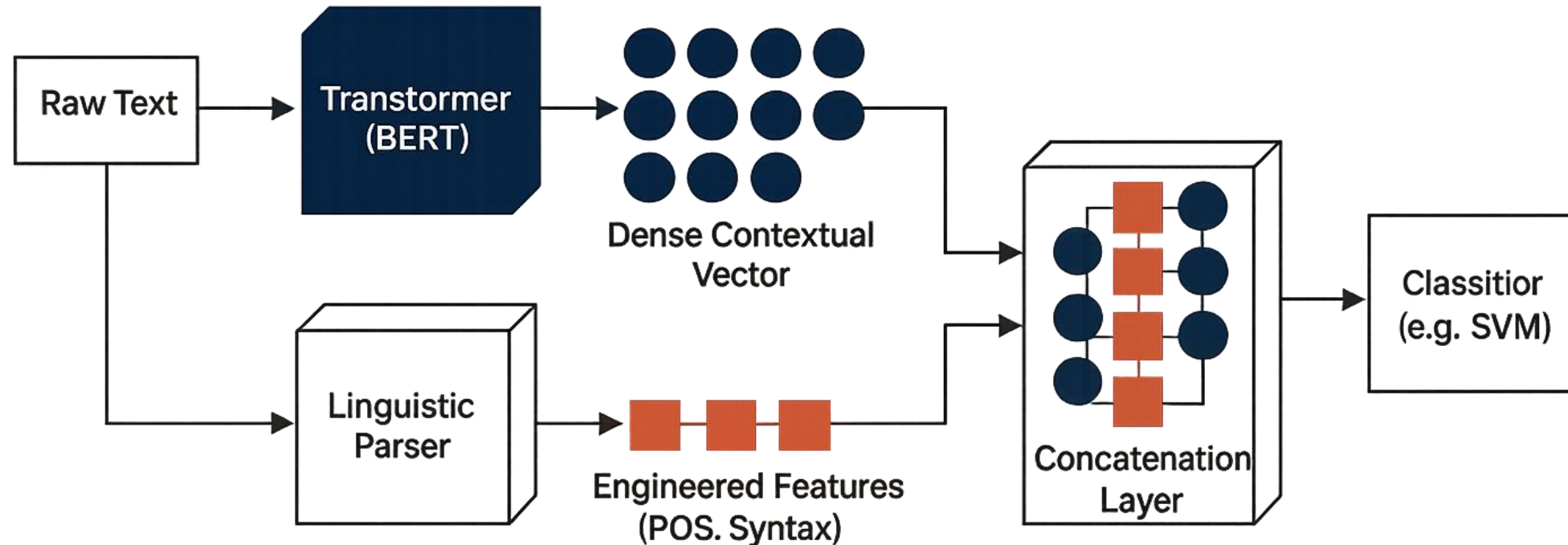


Figure 18: Proposed architectures for CEFR prediction. a) Baseline: original learners' texts as input; b) Concatenation: model taking the original learners' texts and the corrections concatenated; c) Two-streams: model processing the original learners' texts and the corrections with separate streams. (Schmalz and Brutti, 2021)

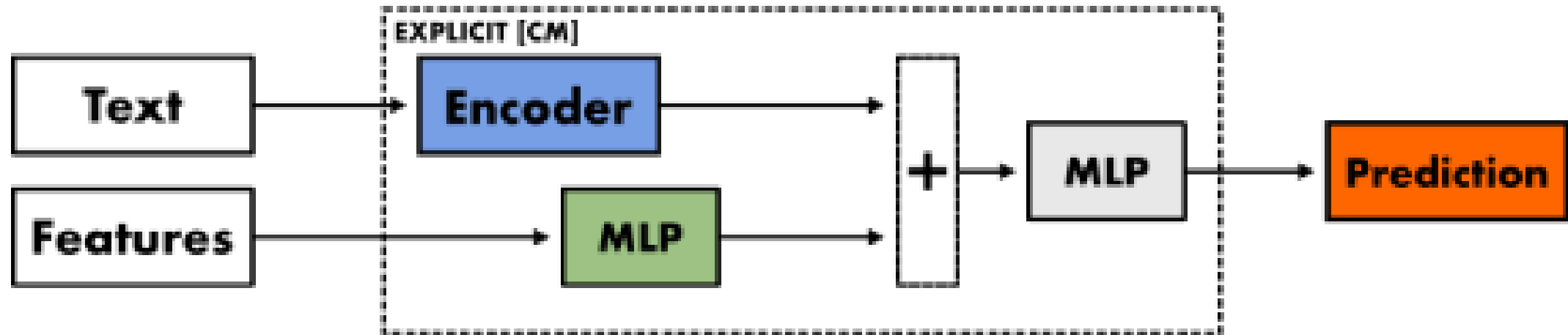
Hybrid Architectures

- Different Modalities (text + table)



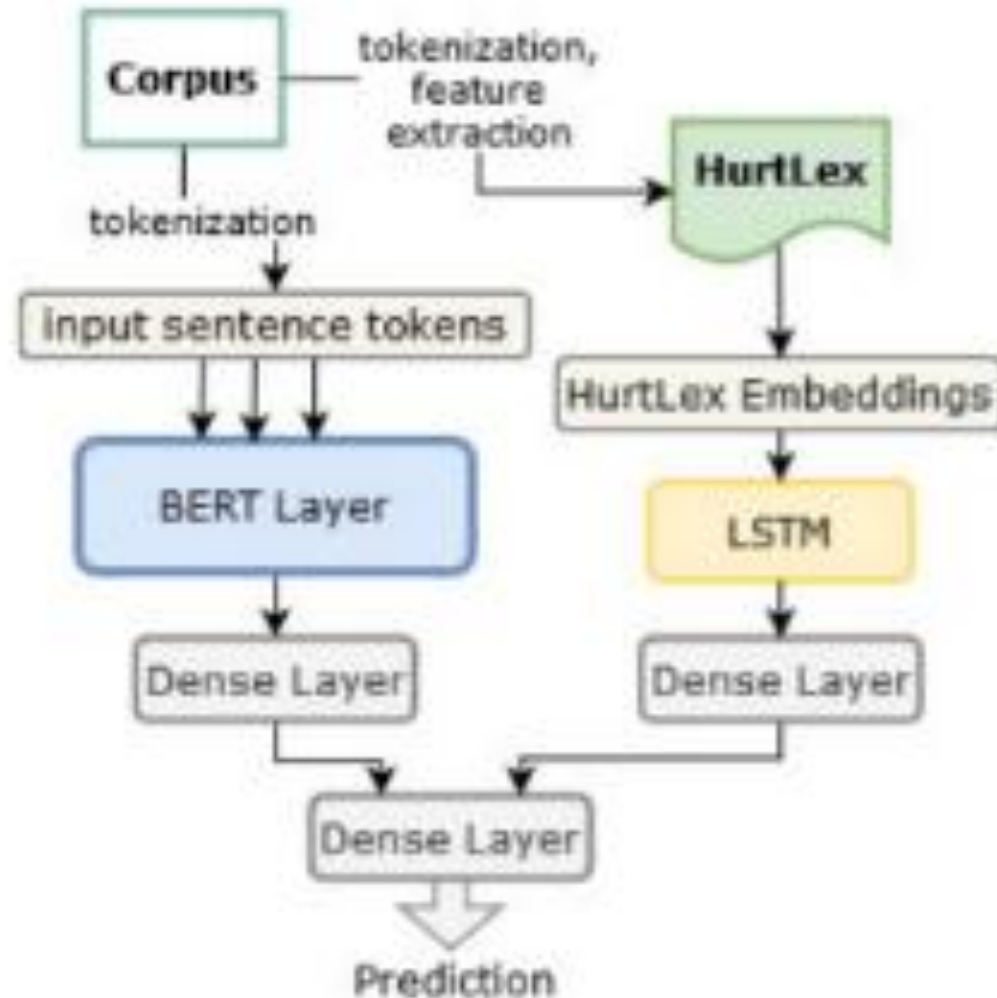
Hybrid Architectures

- Different Modalities (text + table)



Hybrid Architectures

- Different Modalities
 - Text + Dictionary



Hybrid Architectures



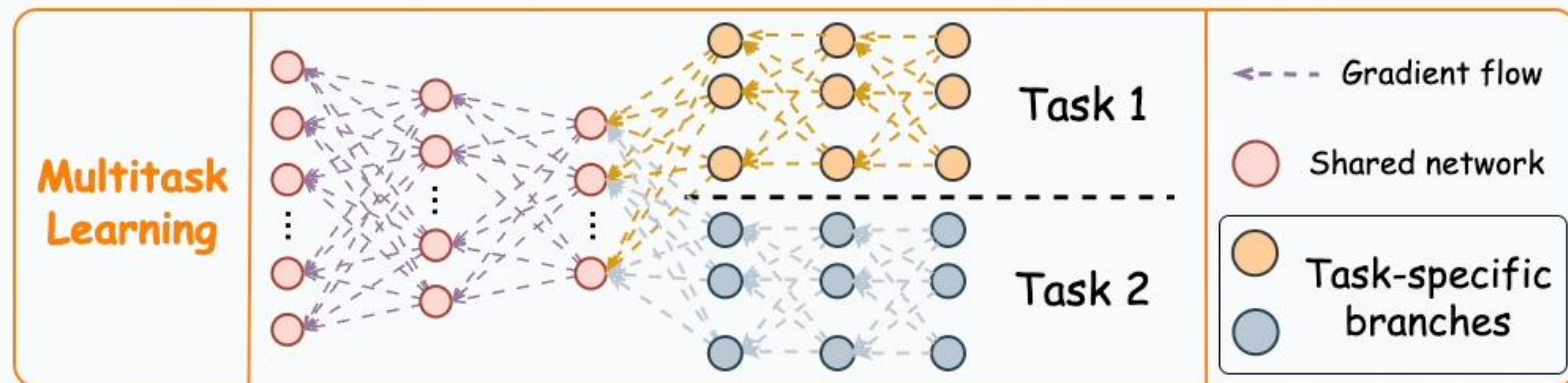
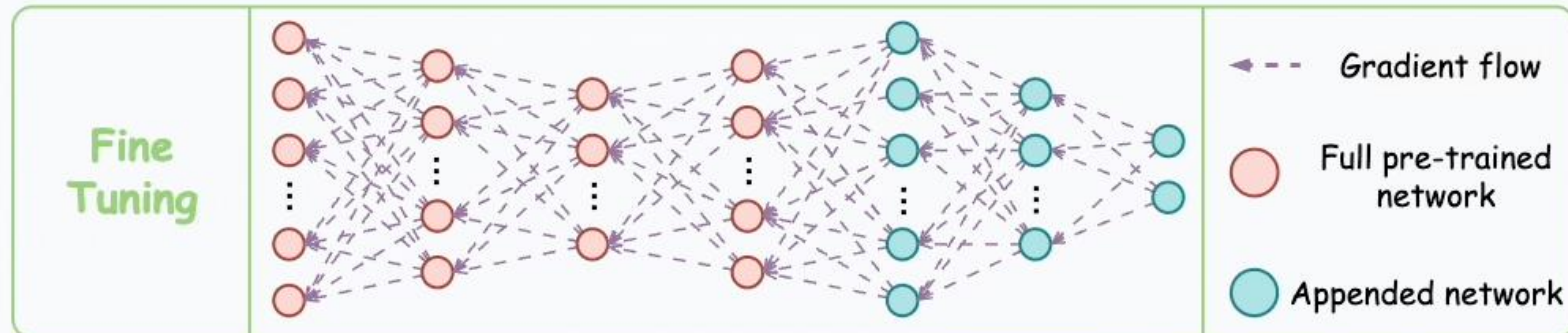
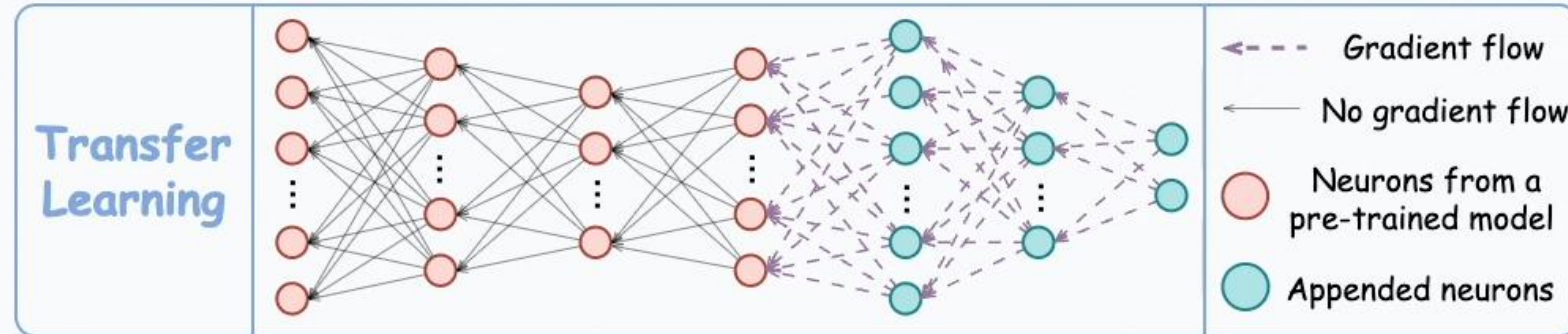
- When would hybrid architectures be useful?

Outline

- Text Encoding
- Classification Approach
- Hybrid Architectures
- **PLM**
- Parameter-Efficient Fine-Tuning

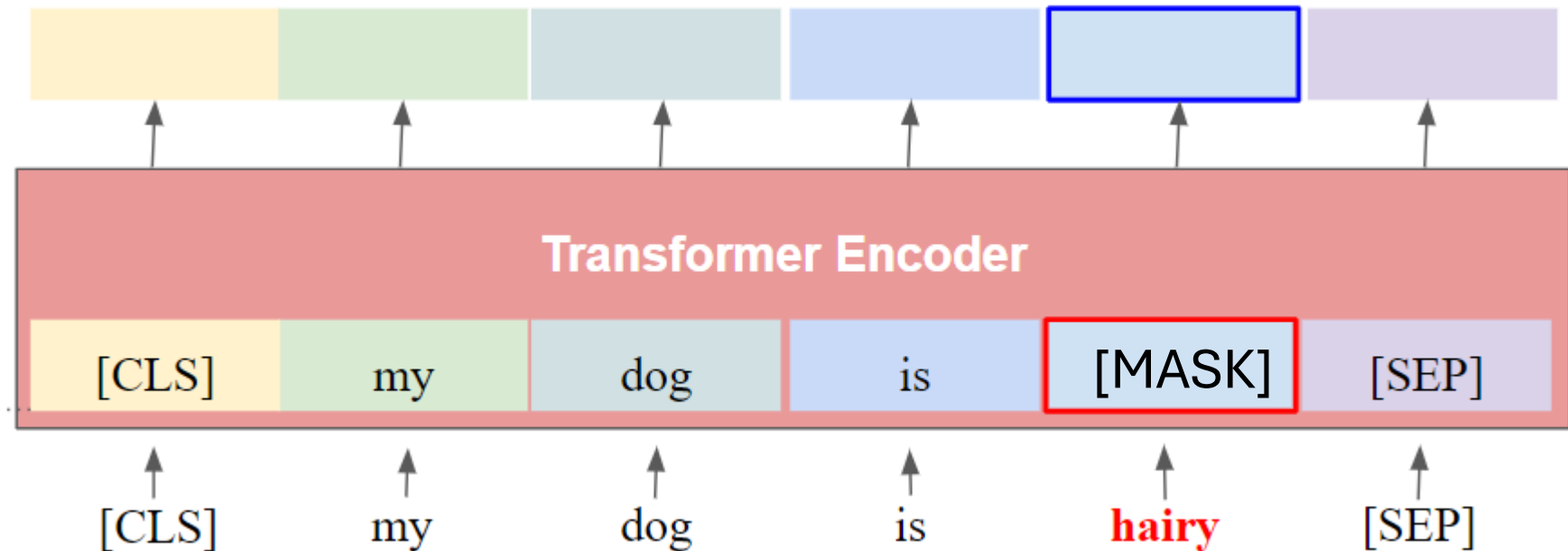
Pretrained Language Model

- PLM



Pretrained Language Model

- PLM
 - Masked Language Modelling (MLM)
 - Cloze task

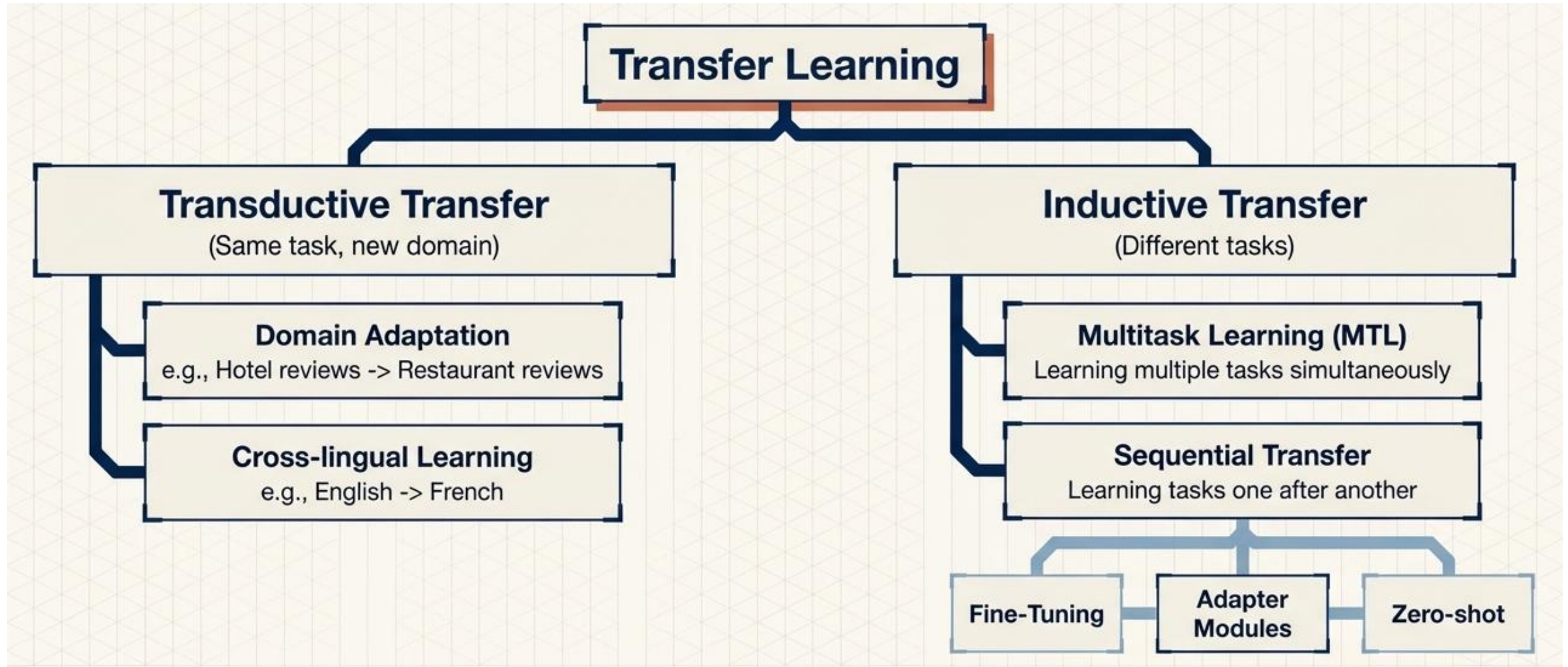


Pretrained Language Model

- PLM

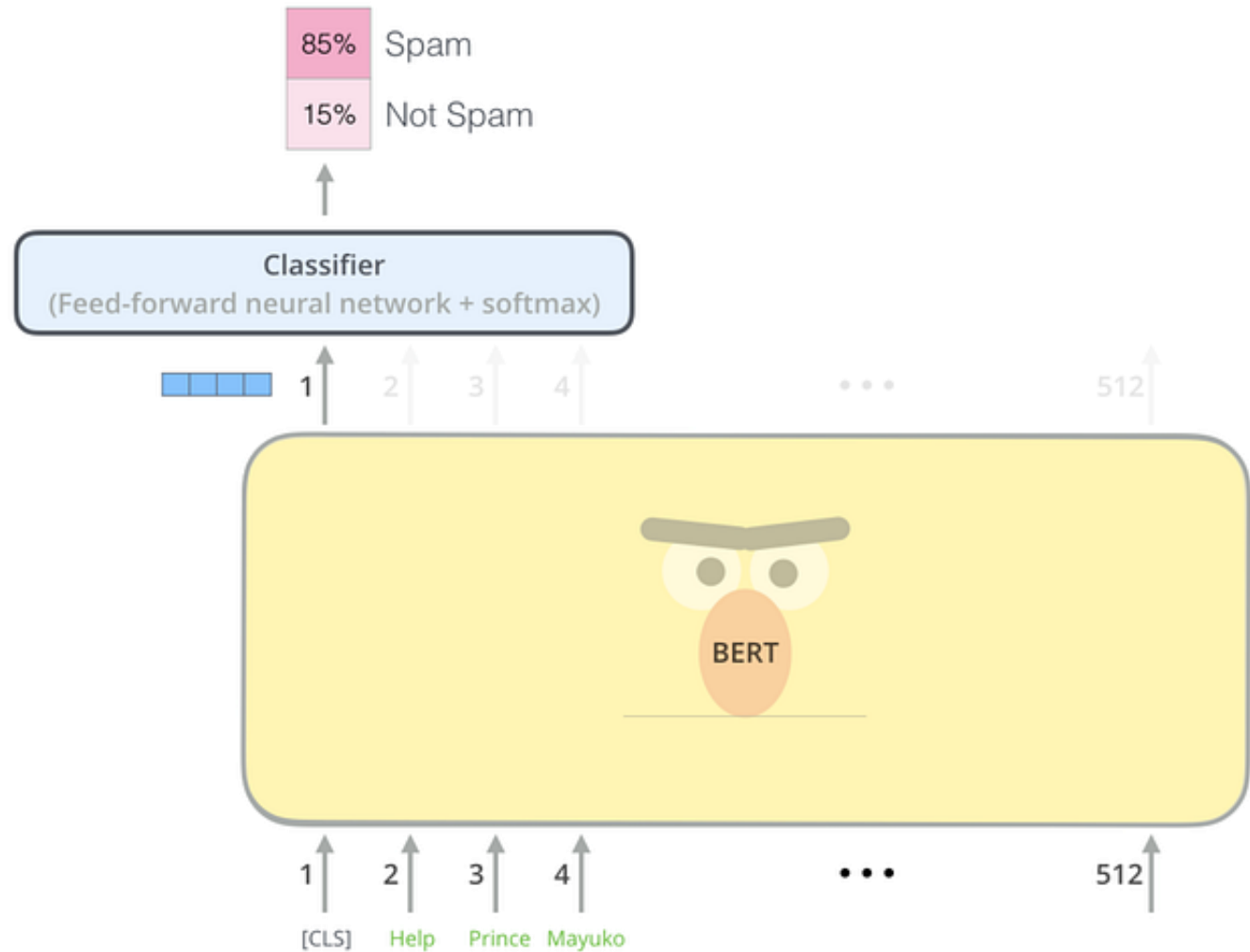
- Next Sentence Prediction (NSP) (Devlin et al., 2019)
- Sentence Order Prediction (SOP) (Lan et al., 2020)
- Sentence Deshuffling (SDS) (Liu et al., 2020b)
- Sentence distance prediction (SDP) (Liu et al., 2020b)
- Replaced Token Detection (RTD) (Xiao et al., 2021)
- ...

Transfer Learning



Transfer Learning

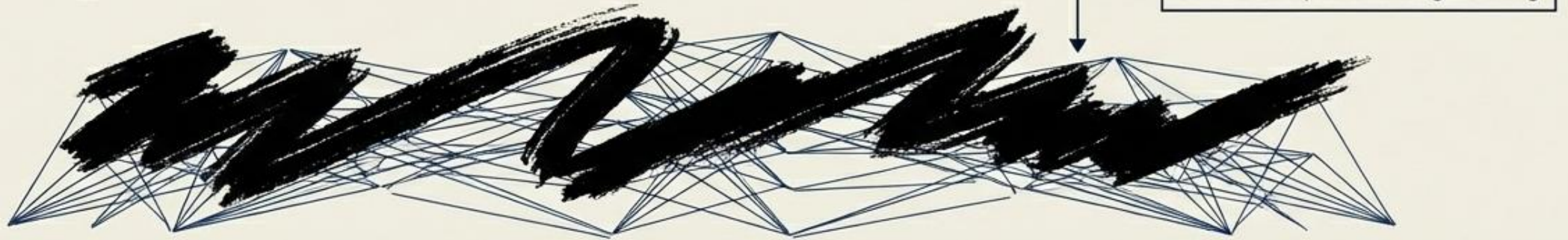
- Fine-tuning



Transfer Learning

- Fine-tuning

Full Fine-Tuning: The Problem



Transfer Learning



- Zero- and Few-shot
 - No training is applied to optimise/learn new parameters
 - GPT

Transfer Learning

- Adapter Modules
 - Instead of updating all parameters, learn only **small additional parameters**
 - The original Transformer parameters remain frozen
 - Only the adapter parameters are trained

Outline

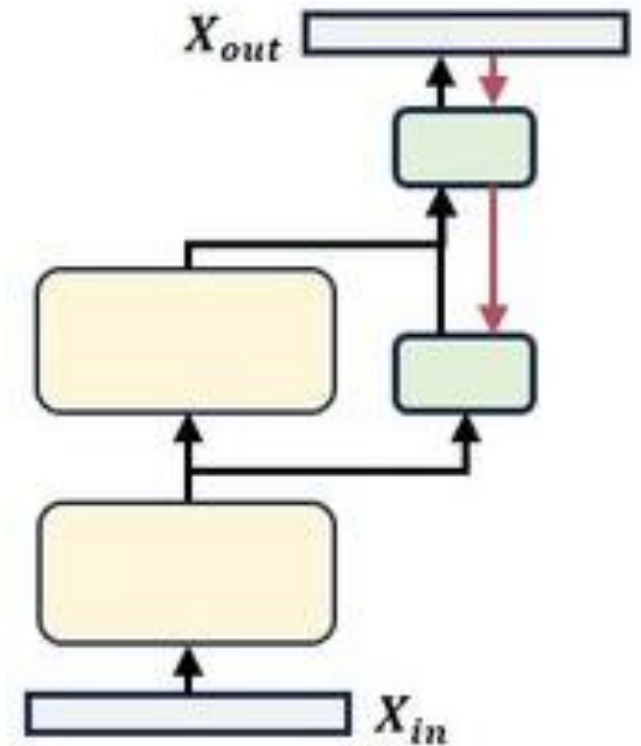
- Text Encoding
- Classification Approach
- Hybrid Architectures
- PLM
- **Parameter-Efficient Fine-Tuning**

Parameter-Efficient Fine-Tuning

- PEFT
 - Bottleneck adapters
 - IA³
 - Prefix Tuning
 - Prompt Tuning
 - Ladder Side-Tuning
 - LoRA

Parameter-Efficient Fine-Tuning

- Ladder Side-Tuning
 - Learn simple representation
 - Like using a frozen model and training the classification head.



→ Forward propagation

→ Backward propagation



Learnable Modules



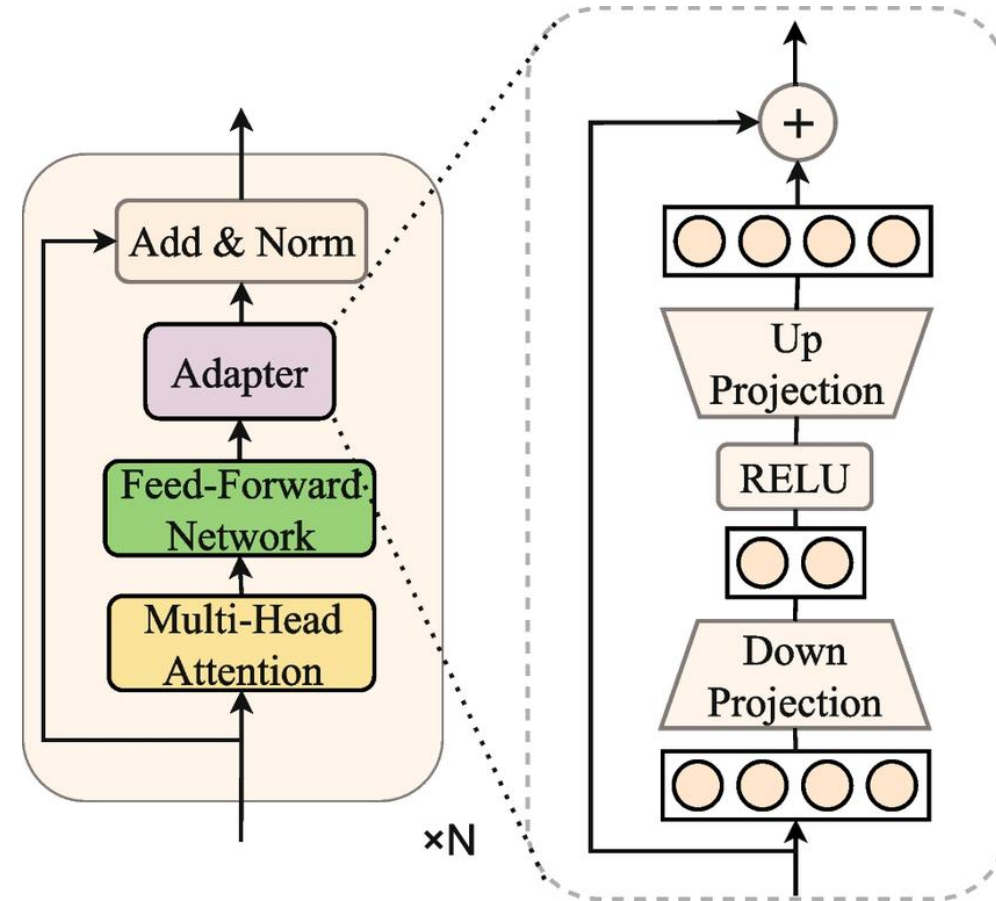
Frozen Modules

Parameter-Efficient Fine-Tuning



University
of Exeter

- Bottleneck adapters
 - Architecture (MLP):
 - Reduce dimensionality
 - Typical bottleneck size: 16,32,48,64
 - Non-linear activation (RELU)
 - Restore original dimensionality
 - Goal:
 - Shift the representation
“a small task-specific correction”

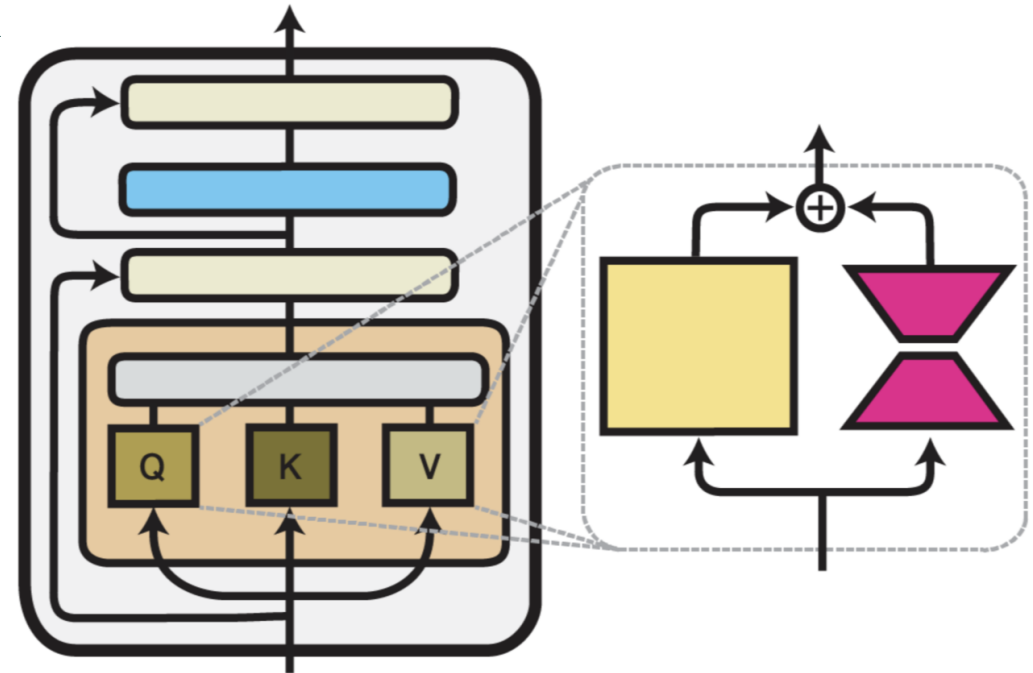


Parameter-Efficient Fine-Tuning

- LoRA (Low-Rank Adaptation)

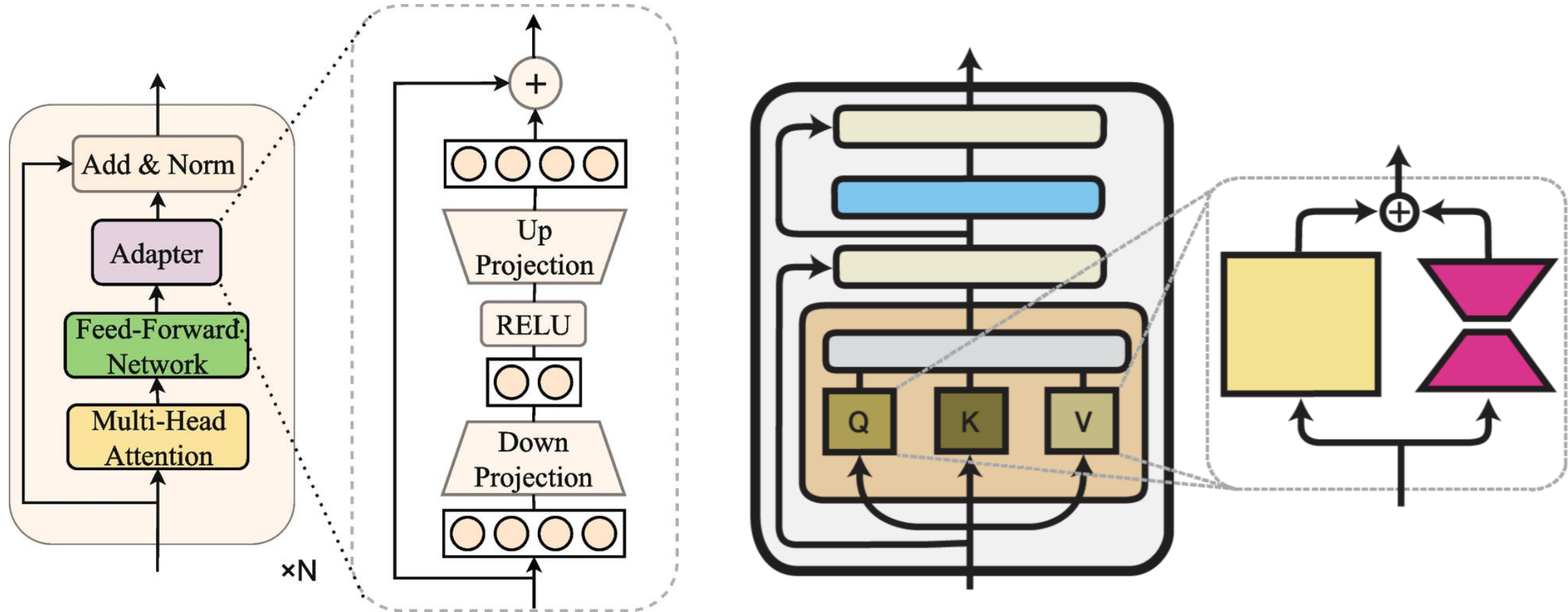
$$\alpha \cdot ((xA)B)$$

- x = input
- A = dimension-reduction matrix
- B = expansion matrix
- α = scaling factor



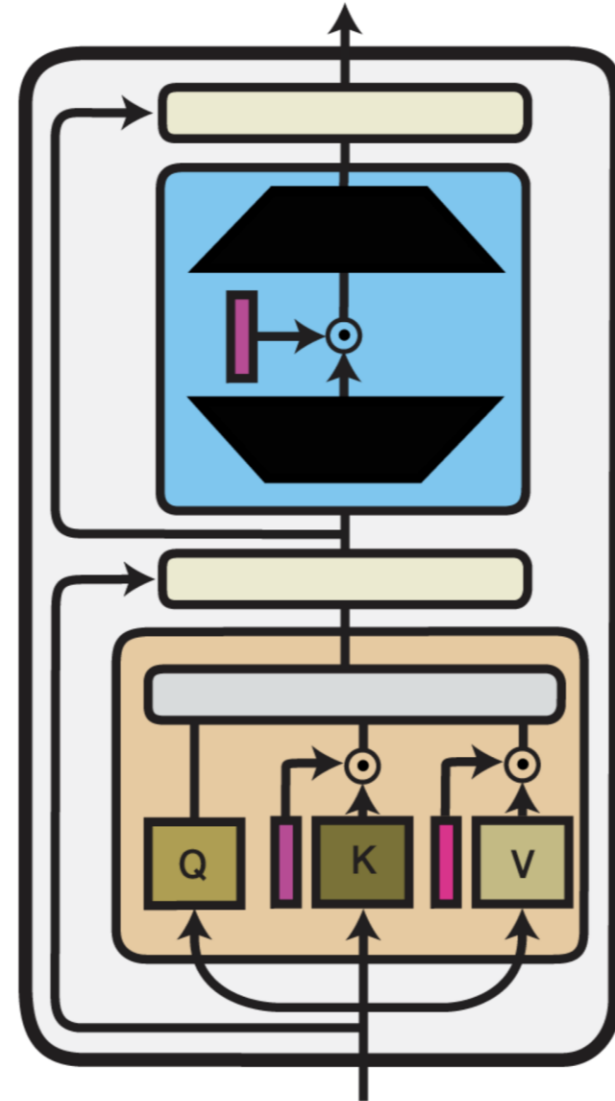
Parameter-Efficient Fine-Tuning

Bottleneck adapters vs LoRA



Parameter-Efficient Fine-Tuning

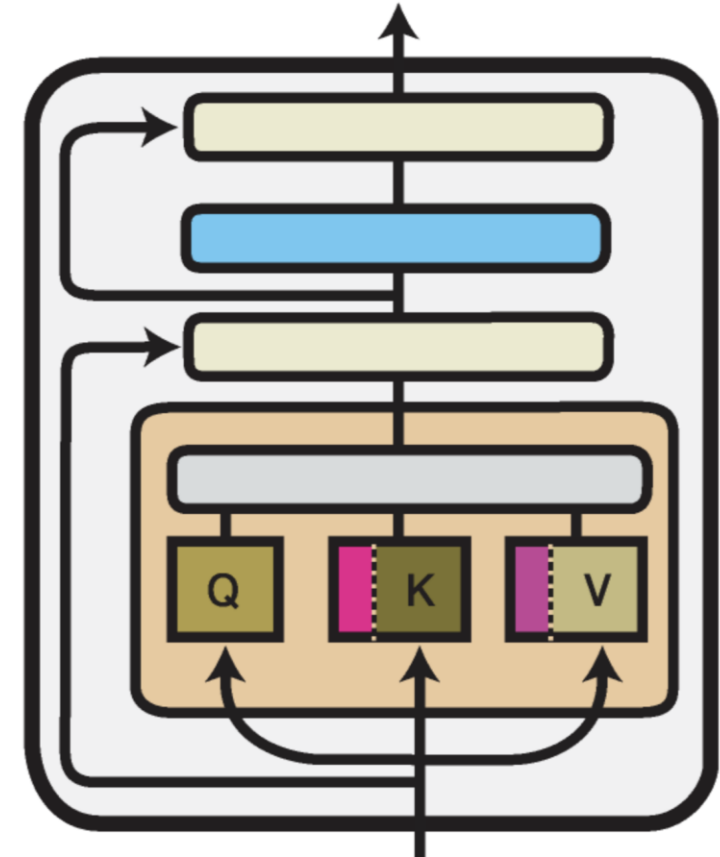
- IA^3
 - (Infused Adapter by Inhibiting and Amplifying Inner Activations)
 - Learn scaling vectors
 - Introduces learnable vectors that scale:
 - attention keys
 - attention values
 - feed-forward activations



Parameter-Efficient Fine-Tuning

- Prefix Tuning

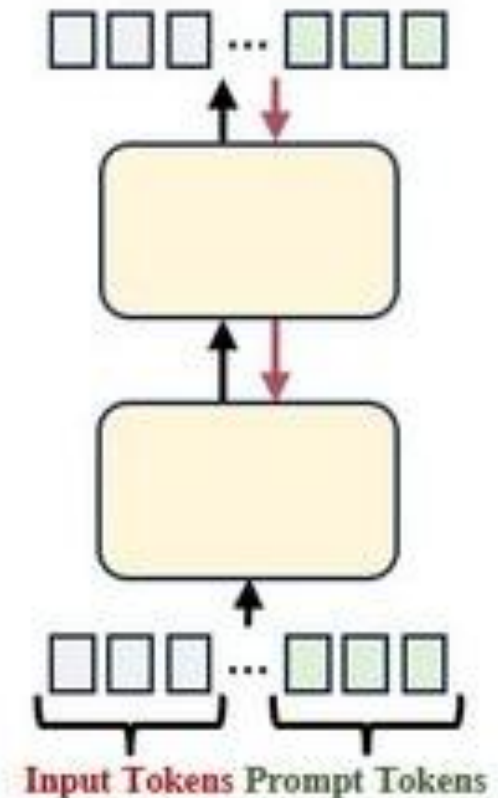
- Learns virtual tokens that influence attention
 - No weight modifications
- Virtual Tokens (vector)
 - TrAnSLAtE
 - ClAssiF SENTiment



Parameter-Efficient Fine-Tuning

- Prompt Tuning

- Learn continuous prompt embeddings
- Training: Only the prompt embeddings are optimised
- Goal: soft task instructions



→ Forward propagation

→ Backward propagation



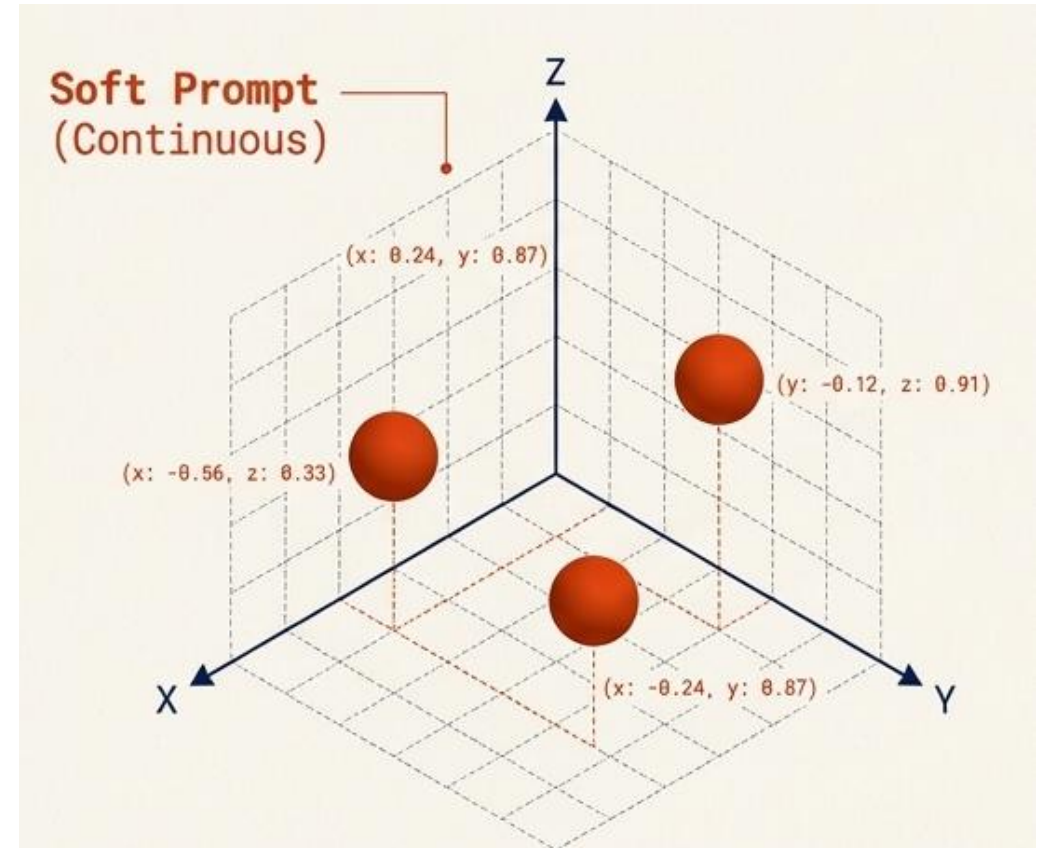
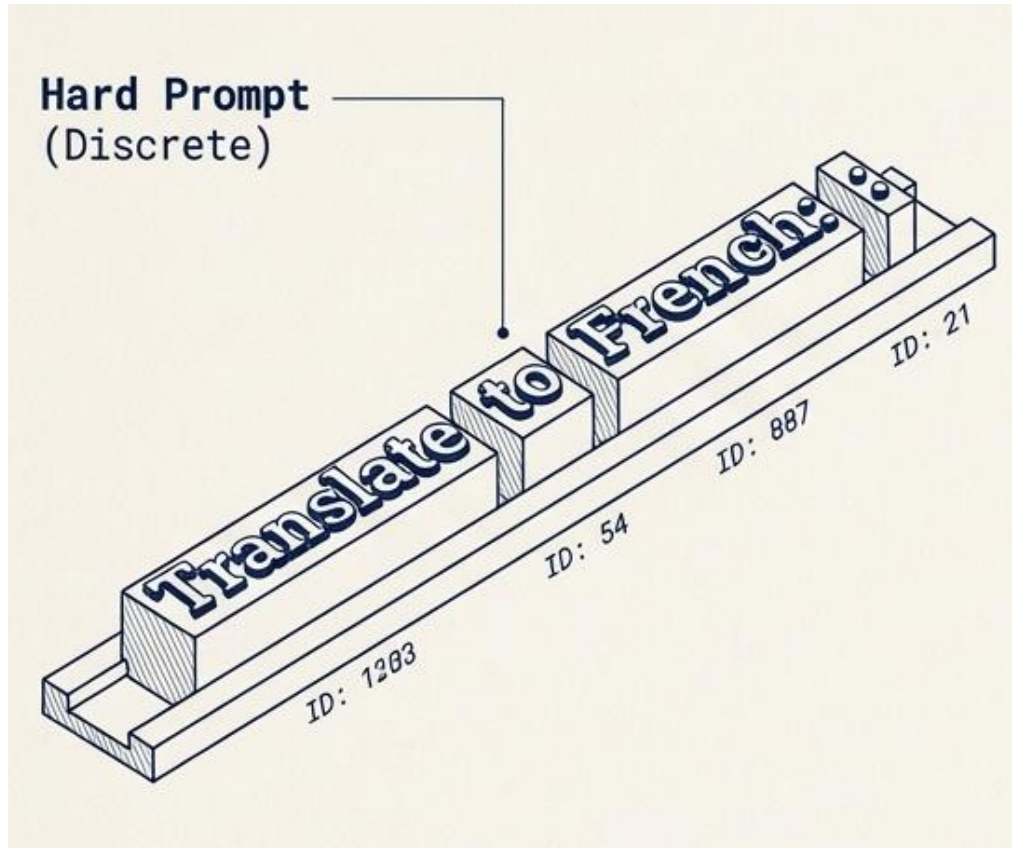
Learnable Modules



Frozen Modules

Parameter-Efficient Fine-Tuning

- Prompt Tuning



Parameter-Efficient Fine-Tuning

Method	Idea	Where parameters are added
Ladder Side-Tuning	small FF network	New layers
Bottleneck adapters	small FF network	transformer layers
LoRA	low-rank weight updates	weight matrices
IA ³	scaling vectors	layer activations
Prefix tuning	learn attention prefixes	attention KV
Prompt tuning	learn soft prompts	input embeddings

Parameter-Efficient Fine-Tuning



- [menti.com](https://www.menti.com)

Conclusion

- By the end of this session, students will be able to:
 - Apply encoding and classification pipelines using count-based, embedding, and transformer representations.
 - Compare count-based, static, sequential, and transformer models regarding context modelling and scalability.
 - Explain PLM pretraining objectives and transfer learning strategies, including domain adaptation.
 - Evaluate parameter-efficient fine-tuning methods (LoRA, adapters, prefix, IA³) for task adaptation.
 - Assess hybrid architectures combining transformers with engineered features or multimodal inputs.